



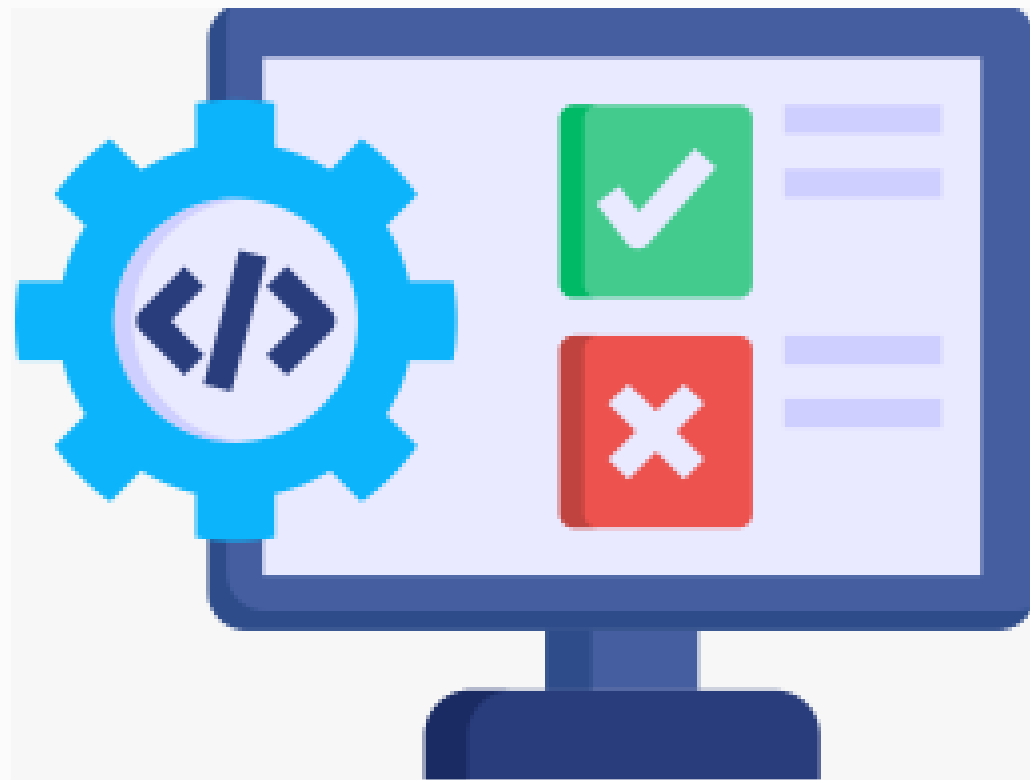
INF352

TESTS ET QUALITÉ DES LOGICIELS

ASSURANCE

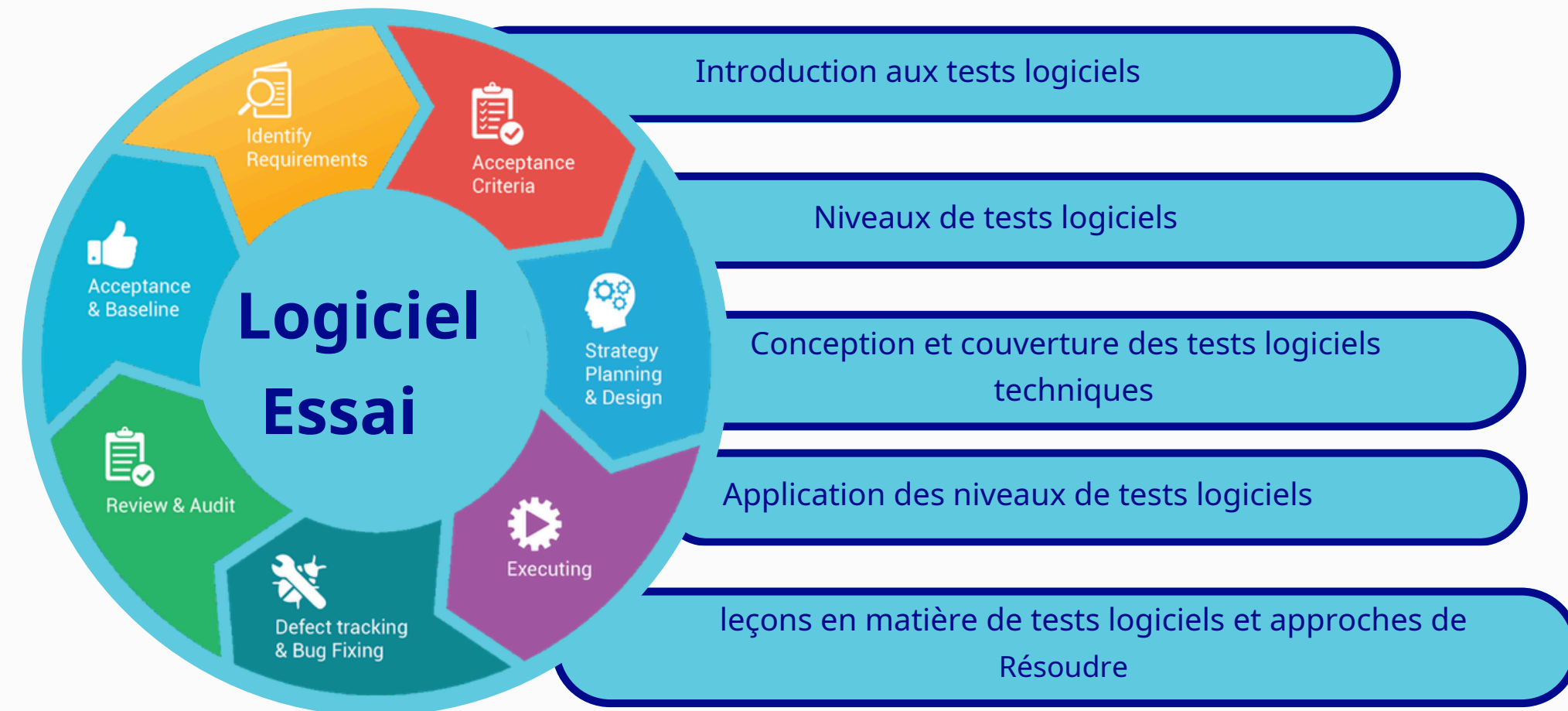


Chargé de cours : Pr. KIMBI Xavéria



Année universitaire.....2025-2026

Planification



Structure du cours

Conférences (CM) :

Approches pédagogiques

Travaux pratiques

Ressources documentaires

Structure de l'évaluation :

- **INF352(EC1) et NF352(EC2)**
- **L'interaction régulière avec les étudiants**
- **invite les praticiens**
- **Séances de réflexion – Après chaque module, demandez aux étudiants d'analyser quelle technique permettra d'éviter les défaillances logicielles réelles.**
- **Démonstration d'outils tels que : JUnit, VTest, Klee et outils de test de mutation**
- **Matériel de cours, documents TD et TP**
- **L'évaluation repose sur l'analyse du code (génération de cas de test), le signalement et le suivi des bogues, ainsi que sur un défi**
- **de test de mutation : score de mutation, nombre de mutants éliminés, couverture des tests, etc.**

Critères d'évaluation :

Présence:

- Pris dans compte -
Participation aux discussions en classe

TD :

- Évaluation de devoirs
exercices
- Petites évaluations basées sur les
exercices du document TD

TP :

- Projet de groupe, recherche et
présentations pratiques

Examens :

- CC(20) + TP(30) + EE(50)

Objectifs du cours

- Notre objectif est de doter les étudiants des compétences et des connaissances nécessaires pour concevoir, mettre en œuvre et exécuter des stratégies de test logiciel efficaces qui détecter défauts, prévenir échec et atténuer risques dans les systèmes logiciels.

À l'issue de ce cours, les étudiants devraient être capables de :

- utiliser efficacement les concepts, méthodologies et techniques de test logiciel pour réaliser des tests logiciels vérification et validation dans des projets concrets.
- Décrivez différentes approches de la vérification et de la validation des logiciels.
- L'étudiant saura comment écrire un script de test et exécuter des tests avec JUnit, Vtest, Jest et Apache JMeter.
- Les étudiants seront capables d'effectuer des tests logiciels en utilisant des logiciels réels.

Contenu du cours

Module 1

- Comprendre les fondamentaux des tests logiciels
- Les rudiments des tests logiciels

Module 2

- Techniques de test statique/boîte noire
- Mise en œuvre d'une étude de cas réelle

Module 3

- Techniques de test dynamiques
- Mise en œuvre d'une étude de cas réelle

Module 4

- Résolution de problèmes dans les tests logiciels et approches pour les résoudre
- Exécution symbolique, tests de mutation : explosion de chemins et génération de cas de test

Module I : Introduction à Tests logiciels

Comprendre les principes fondamentaux des tests logiciels

Qu'est-ce que le test logiciel ?

- Les tests logiciels consistent à exécuter un programme dans le but de vérifier son bon fonctionnement.erreurs . - À garantir que le système fonctionne conformément aux spécifications de conception.
- Les tests logiciels consistent à examiner le comportement d'un système afin d'en découvrir les failles potentielles.défauts
- Les tests logiciels consistent à vérifier si un logiciel est exempt de défauts ou non.

Erreur ==>> Défauts ==>> Échec

- Le SRS est la pierre angulaire du développement logiciel : Pourquoi ? Le client sait ce qu'il veut.

On s'attend à ce que le développeur comprenne ce qu'il doit coder et que le testeur comprenne ce qu'il doit tester.

■

Les tests logiciels constituent la pierre angulaire de tout projet de développement logiciel.

Comprendre les principes fondamentaux des tests logiciels

Définitions de base

- **Erreur (Faute)** Une action humaine qui produit un résultat incorrect.
- **Défaut (anomalie, bug)**: Manifestation d'une erreur dans le logiciel
- **Échec** Le système fonctionne incorrectement. **visible** à l'utilisateur.
- **Cas de test** Un ensemble d'entrées, avec des conditions d'entrée spécifiques et des résultats attendus
- **Suites de tests**: Une collection de cas de test

Comprendre les principes fondamentaux de Tests logiciels

Les logiciels sont partout : de votre téléphone aux dispositifs médicaux en passant par les systèmes bancaires et la finance.

Ces systèmes sont-ils exempts de bugs ?

Les bogues peuvent s'avérer coûteux en raison de tests insuffisants : sur le plan financier, en termes de réputation et même en mettant des vies en danger.

- Transactions frauduleuses dans les banques, les systèmes financiers et de télécommunications.**
- Dysfonctionnement d'appareils contrôlés par logiciel, par exemple des appareils à rayons X, des systèmes d'oxygène, des avions, des factures d'électricité avec des relevés de compteurs falsifiés**

Les bugs et leurs conséquences désastreuses dans la vie réelle :

Comprendre les principes fondamentaux de Tests logiciels

Conséquences de tests logiciels de mauvaise qualité :

Des tests logiciels de mauvaise qualité peuvent avoir des conséquences catastrophiques, entraînant

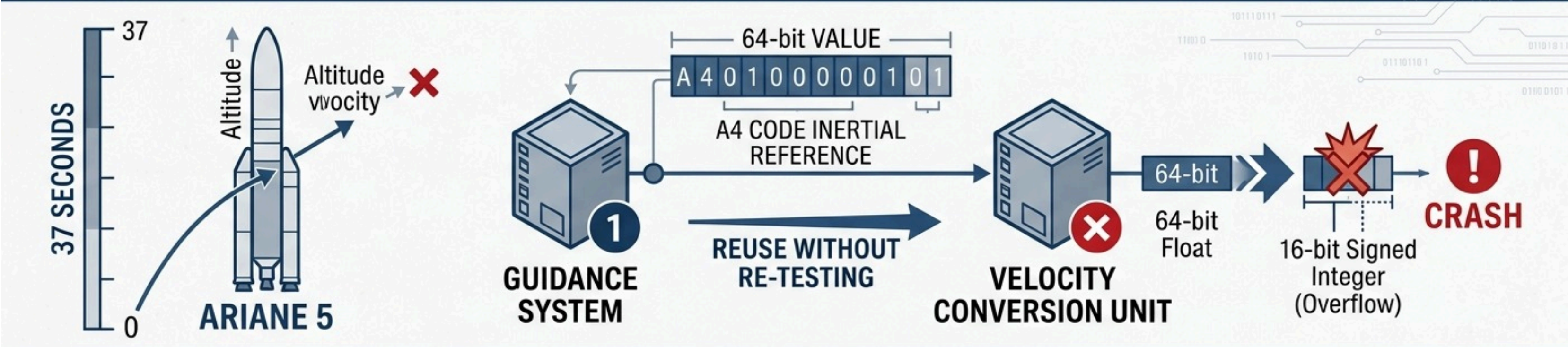
- **Perte de revenus**
- Détérioration de l'expérience utilisateur, atteinte
- à la réputation de l'organisation, pertes en vies
- humaines et matérielles
- Risques pour la sécurité (logiciels critiques pour la
- sécurité) Vulnérabilités de sécurité

Les insectes et leurs conséquences désastreuses dans l'histoire

Échecs logiciels dans l'histoire :Explosion de la fusée Ariane 5 (1996)

Lors du lancement, la fusée a présenté un comportement normal jusqu'à la 37e seconde du vol.

ARIANE 5 FLIGHT V88 SOFTWARE ENGINEERING FAILURE ANALYSIS



LESSONS FOR SOFTWARE ENGINEERING & TESTING

- 1 ROBUST COMPONENT REUSE**
 - Verify assumptions for new operating environments
- 2 FORMAL CODE ANALYSIS**
 - Automated checks for range/precision errors
- 3 FULL SYSTEM INTEGRATION TESTING**
 - Verify components under realistic constraints
- 4 COMPREHENSIVE TEST COVERAGE**
 - Identify and mitigate "Unlikely" failure modes

Comprendre les principes fondamentaux des tests logiciels

Les défaillances logicielles dans l'histoire

Fusée Ariane 5 (1996)

Les raisons :

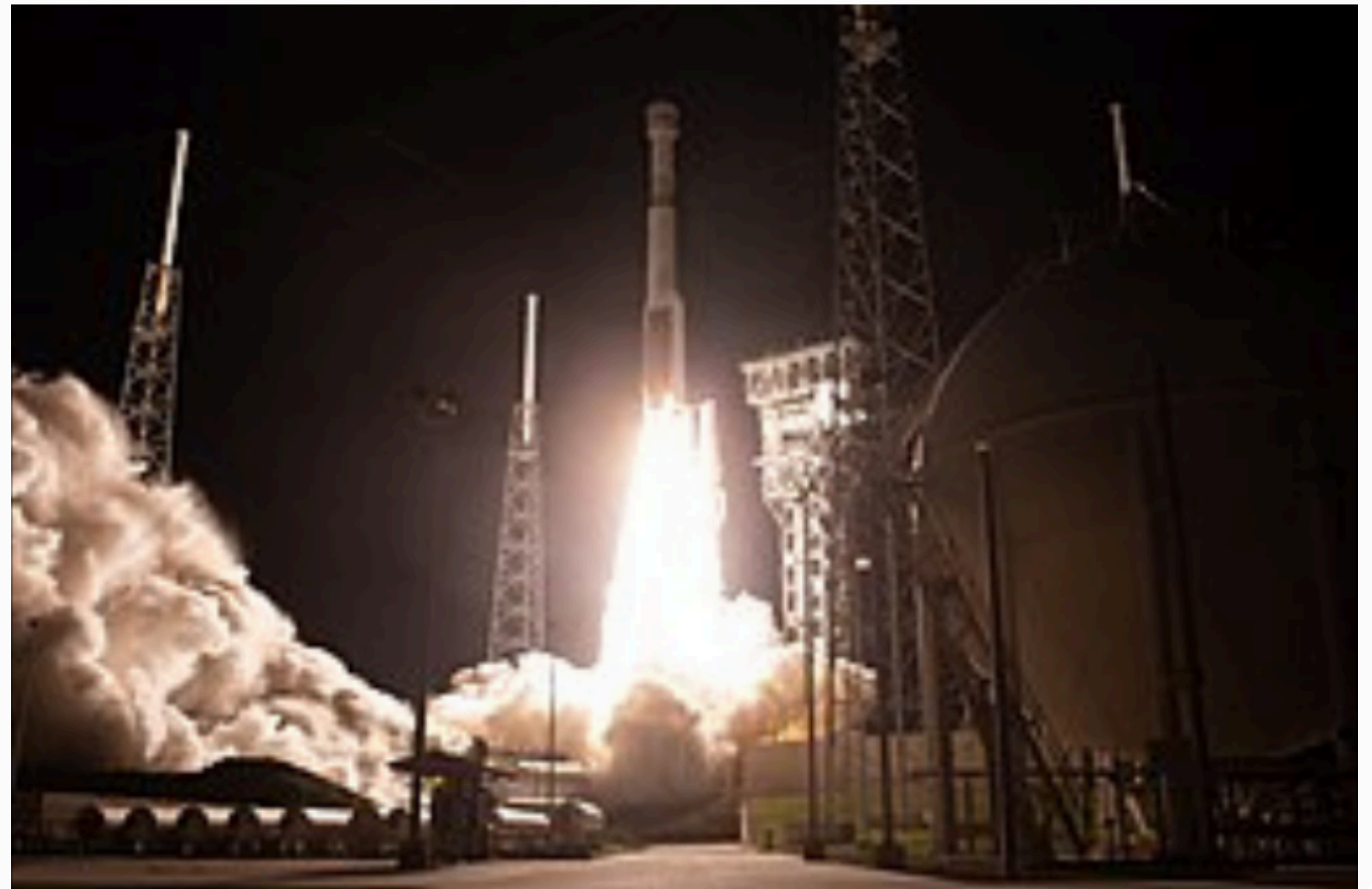
- Un morceau de code d'Ariane 4 (lié au positionnement et à la vitesse de la fusée) a été réutilisé dans Ariane 5.
- Il contenait une conversion d'un nombre à virgule flottante de 64 bits en un entier signé de 16 bits.
- Pour Ariane 5, la valeur à virgule flottante dépassait la valeur maximale pouvant être convertie.
- Panne du système de positionnement.
- La fusée a « corrigé » sa trajectoire.
- Suite à une déviation excessive, Ariane 5 s'est autodétruite !

Comprendre les principes fondamentaux des tests logiciels

Défaillances logicielles dans l'histoire

Essai en vol orbital du Boeing Starliner (2019) :

Lors d'un vol d'essai sans équipage vers la Station spatiale internationale, le vaisseau spatial Starliner de Boeing a rencontré des problèmes logiciels qui l'ont empêché d'atteindre l'orbite prévue, entraînant une fin prématurée de la mission.



Source Wikipédia : Essai en vol orbital de Boeing

Défaillances logicielles dans l'histoire : Histoires vraies d'échecs logiciels

Incident	Le "Bug"	La conséquence
Dispositif d'imagerie médicale - Therac 25	Une « condition de concurrence » dans le logiciel.	La machine a administré aux patients des doses mortelles de radiations car le logiciel n'a pas vérifié si le matériel était prêt.
Knight Capital (2012)	Du code mort laissé en environnement de production a été déclenché accidentellement.	L'entreprise a perdu 440 millions de dollars en 45 minutes, ce qui a entraîné sa faillite.

Comment pouvons-nous vérifier et valider de tels contrôles dans un système logiciel afin d'éviter toute récurrence future ?

⇒ La nécessité de vérifier et de valider les systèmes logiciels.

Tests logiciels et assurance qualité

Comprendre les principes fondamentaux des tests logiciels

Qu'est-ce que la qualité logicielle ?

- Qualité du logiciel désigne le degré auquel un produit logiciel répond aux exigences spécifiées.
répond aux besoins des utilisateurs et fonctionne de manière efficace, sécurisée et fiable dans différentes conditions.
- Assurance qualité permet la mise en œuvre d'un ensemble de mesures qui seront prises tout au long de
les différentes phases du développement logiciel pour augmenter les chances d'obtenir un logiciel qui
répond à ses objectifs (ses spécifications).

Pourquoi les tests logiciels ? L'objectif des tests logiciels

L'objectif général des tests logiciels est de confirmer que qualité des systèmes logiciels par exécuter systématiquement le logiciel dans des circonstances soigneusement contrôlées pour :

- **Révéler les défauts** (Détection des erreurs : erreurs de syntaxe, erreurs logiques, erreurs sémantiques, etc.)
- **Établir la confiance dans le logiciel** - Clarification des spécifications des exigences de l'utilisateur (issues de la phase d'analyse des exigences – exigences fonctionnelles et non fonctionnelles)

S'assurer qu'un produit logiciel répond aux exigences spécifiées, fonctionne correctement et fournit une **expérience utilisateur fluide, fiable et sécurisée**.

- Elle se concentre sur la fourniture de logiciels efficaces, **maintenable, évolutif, et exempt de défauts** tout en s'alignant sur les objectifs commerciaux et les attentes des utilisateurs.

Limites des tests logiciels

- Les tests logiciels ne peuvent que démontrer la présence d'erreurs, et non leur absence.

Pourquoi: Des tests exhaustifs sont impossibles

Défis d'explosion de chemin

Spécifications des exigences incomplètes, informelles et évolutives, ou exigences des utilisateurs incomplètes.

La nécessité de la vérification et de la validation des logiciels

Existe-t-il une différence entre les tests logiciels et le débogage ?

Tests et débogage

Quelle est la différence entre les tests et le débogage ?

- **Essai**: Cela implique l'identification d'un bogue/erreur/défaut dans le logiciel sans le corriger.
- **Débogage** : Cela implique l'identification, l'isolement et la correction (I2F) des problèmes/bugs.

Vérification et validation des logiciels

- Vérification et validation (V&V) L'évaluation est le processus d'évaluation (vérification) d'un système logiciel afin de s'assurer qu'il répond aux exigences caractéristiques et qu'elle remplit ses but prévu On peut également l'appeler Contrôle de la qualité des logiciels

- **Vérification du logiciel** : Il s'agit du processus de vérification visant à garantir que le logiciel répond aux spécifications fonctionnelles. Il s'agit également du processus permettant de déterminer si les produits d'une phase donnée du processus de développement logiciel satisfont aux exigences établies lors de la phase précédente.

Sommes-nous en train de concevoir le produit correctement ?

- **Validation logicielle** : Le processus d'évaluation du logiciel à la fin de chaque phase de développement vise à garantir sa conformité à l'usage prévu, c'est-à-dire à s'assurer que le logiciel répond aux besoins des utilisateurs.

Suis-je en train de concevoir le bon produit ?

- Par conséquent, à chaque étape du cycle de vie du développement logiciel, la vérification et la validation DOIVENT être effectuées.
être effectué.

Comprendre les principes fondamentaux des tests logiciels

Techniques de vérification et de validation de logiciels

- **Analyse statique/Tests de boîte noire**– Raisonnement humain et subjectivité sans formalisme mathématique rigide ni fonctionnement interne complexe : Inspection logicielle (les documents d'exigences, les documents de conception et les plans de test sont tous formellement inspectés par des experts indépendants des développeurs, revue de code, etc.)
- **Tests structurels/dynamiques/en boîte blanche**– {Tests unitaires, tests d'intégration, tests système --> Niveau de tests en boîte blanche, analyse de mutation, exécution symbolique, tests d'oracle, tests événementiels, tests concoliques
 - **Tests en boîte grise**- Un mélange de boîte noire et blanche avec une connaissance partielle du fonctionnement interne
- **Analyse formelle/Méthodes**- Preuves mathématiques formelles utilisant la notation Z, événement Notation B, vérification de modèles, etc.

Comprendre les principes fondamentaux des tests logiciels

Inspections de logiciels

Le processus exige que les documents d'exigences, les documents de conception, le code source et les plans de test soient tous formellement inspectés par des experts indépendants de l'auteur du livrable.

Moderne Logiciel Essai Gestion (Logiciel)
Inspections)

Tests logiciels joue un rôle clé dans la vérification de l'adéquation du logiciel à son usage, et deux types (familles) clés de tests logiciels sont les tests boîte noire et les tests boîte blanche. **boîte blanche** essai.

Boîte blanche Les tests consistent à vérifier que chaque chemin d'un module a été testé et impliquent la définition et l'exécution de cas de test pour garantir la couverture du code et des branches.

L'objectif de **boîte noire** Les tests servent à vérifier la fonctionnalité d'un module ou d'une fonction, ou du système complet lui-même.

Comprendre les principes fondamentaux des tests logiciels

Erreurs de programme / Erreurs logicielles / Bogues logiciels

Les erreurs dans un programme, qu'elles soient dues à des erreurs de conception ou à des erreurs de codage, sont appelées bogues ou défauts.

Erreur On parle d'erreur lorsque le comportement d'un système n'est pas conforme aux spécifications du document de spécifications. Les erreurs de programme sont classées comme suit :

- **Erreurs de syntaxe** : Ces erreurs surviennent lorsque le code enfreint les règles grammaticales du langage de programmation. Le compilateur ou l'interpréteur ne peut alors pas comprendre le code. Par exemple : oublier un point-virgule en fin de ligne en C++, ou écrire ``prnit()`` au lieu de ``print()`` en Python.
- **Erreurs logiques** : Erreurs de spécification, de conception et d'implémentation. Elles surviennent lorsque le programme s'exécute sans planter, mais produit un résultat erroné. La logique de l'algorithme est alors défectueuse. Par exemple : utiliser un `+` au lieu d'un `*` pour calculer une aire, ou une erreur d'incrémentation où une boucle s'exécute 10 fois au lieu de 11.

•

Comprendre les principes fondamentaux des tests logiciels

Erreurs de programme / Erreurs logicielles / Bogues logiciels

Erreurs d'exécution : Ces erreurs surviennent pendant l'exécution du programme, souvent en raison de facteurs environnementaux inattendus ou de données invalides que la vérification syntaxique n'a pas pu détecter.

Exemple : Division par zéro, tentative d'ouverture d'un fichier inexistant ou « dépassement de capacité de la pile » dû à une récursion infinie.

- **Erreurs sémantiques** : Ces erreurs se produisent lorsque les instructions sont syntaxiquement correctes mais n'ont pas de sens dans le contexte de la logique ou des types de données du programme.

- **Exemple** : Tentative d'addition d'une chaîne de caractères à un entier (« Âge » + 25) ou utilisation d'une variable avant qu'elle n'ait été correctement initialisée à des fins spécifiques.

- Etc

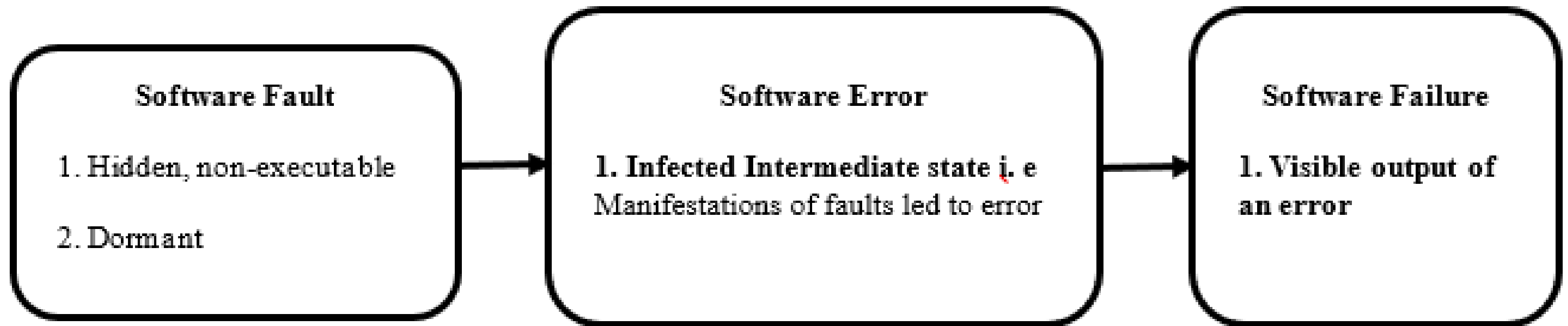
- D'après l'explication ci-dessus, quelles sont les différences entre un défaut logiciel, une erreur logicielle et une panne logicielle ?

Question (1) : Énumérez trois catégories d'erreurs de programme avec cinq (5) exemples chacune.

Question (2) : Citez trois causes d'erreur de programme.

Comprendre les principes fondamentaux des tests logiciels

Relation entre les défauts de programme, les erreurs de programme et les défaillances de programme



Software Defects Relationships

Comprendre les principes fondamentaux des tests logiciels

Défaut ≠ Anomalie

Exemple : Une anomalie (telle qu'une maladie) peut toujours être expliquée par un défaut (un pathogène), alors qu'un défaut (un microbe latent) ne provoquera pas nécessairement une anomalie.

Les tests intervenant après la programmation, nous ne nous intéressons ni aux erreurs humaines déjà commises, ni à leur prévention. Notre objectif est de corriger les défauts introduits involontairement afin de minimiser les anomalies potentielles.

Sans incidence sur le reste de ce cours, nous pourrions — par un mauvais usage du langage — confondre «erreur**» et "**défaut**"(une tendance humaine à confondre **cause** et **conséquence**!).**

Comprendre les principes fondamentaux des tests logiciels

Catégories de défauts

L'ensemble de tous les défauts susceptibles d'affecter un logiciel est infini...

Cependant, on peut identifier des classes spécifiques de défauts : **calcul**, **logique**, **E/S**(Entrée/Sortie), **informatique**, **interface**, **définition des données**, etc.

Les moyens de détecter **défauts** Elles peuvent être automatiques ou manuelles, et elles s'appliquent à la fois au code source et à son comportement.

Comment définir **l'activité de test** dans le cadre d'un projet logiciel ?

Comprendre les principes fondamentaux des tests logiciels

Résumé des leçons

Importance des tests approfondis : Des tests complets et rigoureux des systèmes logiciels sont essentiels pour identifier et corriger les problèmes potentiels avant leur déploiement. Ces tests doivent couvrir divers scénarios et cas limites afin de minimiser les risques de défaillances critiques.

Expérience utilisateur et Intégration des commentaires : Prendre en compte les commentaires des utilisateurs et se concentrer sur l'expérience utilisateur lors du développement logiciel peut aider à identifier les problèmes potentiels avant qu'ils ne deviennent des problèmes généralisés.

Comprendre les principes fondamentaux des tests logiciels

Définitions

extrait de « Tests logiciels » [SX-PR-CK-2000] : **Tests logiciels** est une activité qui fait partie du processus de développement. Elle est menée conformément à **assurance des tests**. Elle définit les règles et commence une fois l'activité de programmation terminée. Elle se concentre à la fois sur le code source et sur le comportement du logiciel. Son objectif est de **minimiser les risques d'apparition d'une anomalie**, en utilisant des moyens automatiques ou manuels visant à détecter à la fois les diverses anomalies possibles et les défauts potentiels qui pourraient les provoquer.

Comprendre les principes fondamentaux des tests logiciels

Définitions

Définition (tirée de la norme IEEE-STD729, 1983) : **Essai** Il s'agit d'un processus manuel ou automatique visant à établir qu'un système vérifie les propriétés requises par sa spécification ou à détecter les différences entre les résultats générés par le système et ceux attendus par la spécification.

De la part de l'AFCIQ : **Essai** est une technique de contrôle consistant à garantir, par son exécution, que le comportement d'un programme est conforme à des données préétablies.

AFCIQ : Association française pour le contrôle et les essais industriels

extrait de « L'art des tests logiciels » [GJM] : **Essai** est le processus de Exécuter un programme dans le but de trouver des anomalies ou des défauts.

Remise à niveau en génie logiciel

Qu'est-ce que le génie logiciel ?

La définition de la norme IEEE 610.12 est la suivante :

génie logiciel « Il s'agit de l'application d'une approche systématique, rigoureuse et quantifiable au développement, à l'exploitation et à la maintenance des logiciels ; c'est-à-dire l'application de l'ingénierie aux logiciels et l'étude de ces approches appliquées aux logiciels. »

Défi

Le défi du génie logiciel est de livrer aux clients des logiciels de haute qualité, soumis à des tests rigoureux, dans les délais et le budget impartis.

Remise à niveau en génie logiciel

Processus et cycle de vie du logiciel

Plusieurs cycles de vie bien connus sont utilisés, tels que le modèle en cascade, le modèle en spirale, le processus unifié rationnel (RUP) et la méthodologie Agile.

Le choix d'un cycle de vie de développement logiciel particulier est déterminé par les besoins spécifiques du projet.

- **Cycle de vie en cascade**
- **Cycle de vie en spirale**
- **Développement agile**

Remise à niveau en génie logiciel

Processus et cycle de vie du logiciel

- Cycle de vie en cascade

Le modèle en cascade commence par la collecte et la définition des exigences. Il est suivi par la spécification du système (des exigences fonctionnelles et non fonctionnelles), la conception et la mise en œuvre du logiciel, et des tests complets.

Les tests comprennent généralement des tests unitaires, système et d'acceptation utilisateur. Le modèle en cascade est utilisé pour les projets dont les exigences peuvent être identifiées tôt dans le cycle de vie du projet ou sont connues à l'avance.

Remise à niveau en génie logiciel

Processus et cycle de vie du logiciel

- Cycle de vie en cascade

Le modèle en cascade commence par la collecte et la définition des exigences. Il est suivi par la spécification du système (des exigences fonctionnelles et non fonctionnelles), la conception et la mise en œuvre du logiciel, et des tests complets.

Les tests comprennent généralement des tests unitaires, système et d'acceptation utilisateur. Le modèle en cascade est utilisé pour les projets dont les exigences peuvent être identifiées tôt dans le cycle de vie du projet ou sont connues à l'avance.



Remise à niveau en génie logiciel

Processus et cycle de vie du logiciel

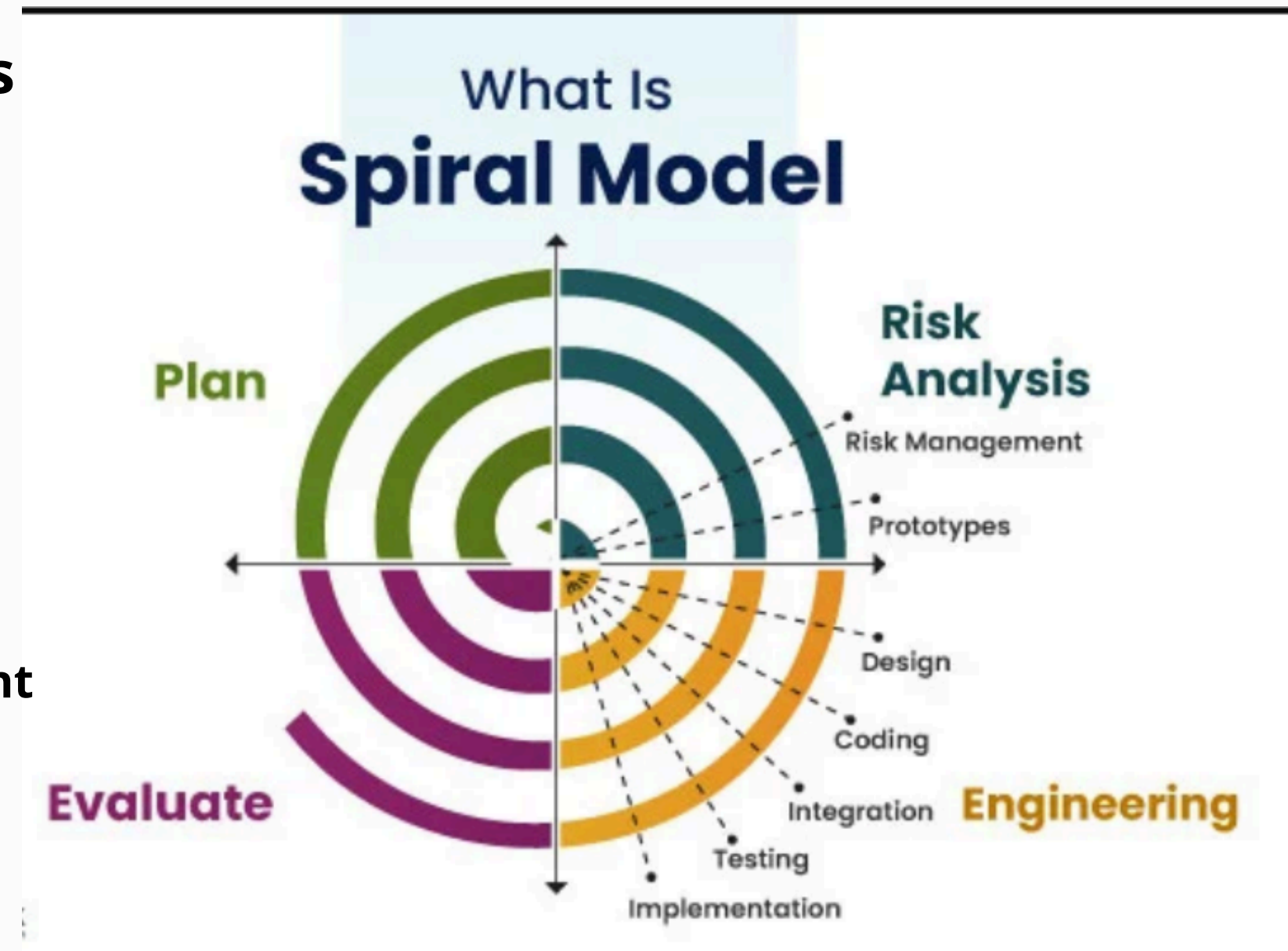
- Le modèle en spirale

Le modèle en spirale est utile pour les projets dont les exigences ne sont pas entièrement connues au début du projet, ou dont les exigences évoluent au cours du développement.

cycle de vie.

Le développement se déroule en plusieurs spirales, chaque spirale

met en œuvre de nouvelles fonctionnalités et implique généralement la détermination des objectifs et analyse des risques, mises à jour des exigences, conception, code, tests et utilisateur examen de l'itération ou spirale particulière



Principes fondamentaux des tests logiciels

Débogage vs Tests

Bien que souvent utilisés de manière interchangeable, **Essai** et **Débogage** sont deux étapes distinctes du cycle de vie du développement logiciel.

Pour utiliser une métaphore médicale : les tests correspondent au bilan de santé ou au diagnostic (la recherche des symptômes), tandis que le débogage correspond à l'intervention chirurgicale ou au traitement (la correction de la cause sous-jacente).

Fonctionnalité	Essai	Débogage
Définition	Le processus de recherche de bogues ou d'erreurs.	Le processus d'isolement et de correction de ces bogues.
But	Pour vérifier si le logiciel répond aux exigences et pour détecter les défaillances.	Trouver la cause profonde d'une panne et la résoudre.
Timing	Une phase planifiée qui se déroule avant et après le débogage.	Un processus réactif qui se produit après l'échec d'un test.
Interprété par	Testeurs (assurance qualité), scripts automatisés ou développeurs.	Plus précisément, les développeurs qui ont écrit ou qui maintiennent le code.
Résultat	Un rapport indiquant « Réussite » ou « Échec » (Confirmation d'un bug).	Un correctif de code et un système restauré (Suppression d'un bug).

Débogage vs Tests

Test : « Est-ce que ça fonctionne ? »

Les tests consistent en une tentative systématique de mettre le logiciel à l'épreuve ou d'en découvrir les limites. Il s'agit de validation et de vérification.

- **Activités clés :**

- **Tests unitaires : Tests effectués de manière isolée sur des fonctions ou des composants individuels.**
 - **Tests d'intégration : vérification du bon fonctionnement conjoint des différents modules.**
 - **Tests en boîte noire : tester le logiciel sans examiner le code source. code interne (axé sur les entrées et les sorties).**
- **Résultat : Vous savez qu'une fonctionnalité spécifique est défectueuse (par exemple, « La connexion »). Le bouton ne fonctionne pas lorsque le mot de passe contient un caractère spécial).**

Débogage vs Tests

Débogage : « Pourquoi est-ce que ça ne fonctionne pas ? »

Débogage Il s'agit d'un processus d'investigation. Il commence lorsqu'un test échoue et se termine lorsque le code est corrigé et que la correction est vérifiée.

- **Le flux de travail de débogage :**

- a. **Reproduction**: Trouver les étapes exactes pour reproduire le bug.

- b. **Localisation**: Réduire les milliers de lignes de code à la ligne précise qui provoque l'erreur.

- c. **Analyse**: Comprendre l'erreur logique (par exemple, « Oh, j'ai utilisé un `>` au lieu d'un `>=` »).

- d. **Correction**: Rédiger le correctif.

- e. **Validation**: Relance du test initial pour s'assurer que le bug a bien été corrigé.

- **Outils courants**: Points d'arrêt, exécution pas à pas, traces de pile et instructions d'impression (`console.log`, `printf`).

Débogage vs Tests

Le cycle relationnel

Essai et Débogage sont les deux faces d'une même pièce. Elles fonctionnent en boucle continue :

Test : Vous exécutez une suite de tests.

Échec : Un test échoue, indiquant la présence d'un bug.

Débogage : Vous examinez le code pour trouver la cause première.

Correction : Vous modifiez le code pour éliminer le bug.

Nouveau test : vous exécutez à nouveau le test pour vous assurer que le correctif fonctionne et n'a pas introduit de « régressions » (nouveaux bugs).

Débogage vs Tests

Debugging:

The process of identifying the cause of an error and correcting it

Testing:

The process of detecting errors during development

Refactoring:

The process of restructuring source code to improve its operation without altering its functionality

Diagnostics:

Instrumenting applications to gather data about its internal operation, after it's been released

Vérification et validation des logiciels

Vérification et validation Le contrôle des tests logiciels, souvent intégré au processus global de contrôle des tests logiciels, consiste à s'assurer qu'un système est conforme à ses spécifications techniques et répond aux attentes de l'utilisateur final.

Vérification et validation des logiciels

1. Définition des concepts fondamentaux

Bien que souvent utilisés de manière interchangeable, la vérification et la validation représentent deux perspectives distinctes de l'assurance des tests :

- **Vérification** Ce volet porte sur la cohérence interne du processus de développement. Il évalue si le résultat d'une phase spécifique répond aux exigences définies lors de l'étape précédente.

Question clé : "**Sommes-nous en train de concevoir le produit correctement ?**"

Priorité : Respect des spécifications et des normes techniques.

- **Validation** Elle se concentre sur les exigences externes et l'utilité finale du logiciel. Elle évalue le système à la fin du développement (ou d'une phase) afin de s'assurer qu'il répond à son usage prévu dans le monde réel.

Question clé : "**Sommes-nous en train de construire le bon produit ?**"

Objectif : Répondre aux besoins des clients et atteindre les objectifs opérationnels.

Vérification et validation des logiciels

2. Analyse comparative : vérification vs validation

Fonctionnalité	Vérification	Validation
Objectif principal	Vérifiez que le logiciel est conforme à sa conception et à ses spécifications.	S'assurer que le logiciel répond aux exigences des utilisateurs et aux objectifs de l'entreprise.
Niveau d'activité	Activité de faible niveau: Concerne la logique, l'architecture et la structure du code.	Activité de haut niveau: Concerne le comportement global et la facilité d'utilisation du système.
Focus sur les données	« Est-ce que j'accède correctement aux données ? » » (Exactitude technique).	« Ai-je accès aux données correctes ? » » (Pertinence fonctionnelle).
Méthodes	Inclut les examens, les visites guidées, les inspections et le respect des listes de contrôle.	Inclut les tests fonctionnels, les tests bêta et les tests d'acceptation.
Timing	Réalisé au cours du développement processus (en cours).	Effectué sur le produit fini ou les principaux modules (Post-processus).

Vérification et validation des logiciels

3. Objectifs stratégiques de V&V

- **L'objectif de la vérification**: Identifier et atténuer de manière proactive les défauts cachés, les incohérences et les erreurs logiques le plus tôt possible dans le cycle de vie.
- **Objectif de la validation**: Afin d'instaurer la confiance dans le logiciel, il convient de démontrer aux développeurs et aux parties prenantes que le système est « adapté à l'usage prévu » et qu'il répond aux exigences initiales.

Vérification et validation des logiciels

4. Méthodologies de vérification et de validation

Les techniques de vérification et de validation sont classées en quatre approches principales en fonction de leur niveau de rigueur et de leur mise en œuvre :

- **Analyse informelle** : Ces méthodes s'appuient sur l'intuition humaine et le jugement subjectif plutôt que sur des règles mathématiques strictes. Les pratiques courantes incluent les revues par les pairs, les présentations et les inspections informelles où les développeurs discutent de la logique et de la conception.
- **Analyse statique** : Cela consiste à évaluer les artefacts logiciels sans exécuter le code. L'objectif est d'examiner la documentation, les spécifications fonctionnelles, les modèles et le code source afin de déceler les défauts structurels ou les non-conformités aux normes. Par exemple, un algorithme analyse et vérification du code.

Vérification et validation des logiciels

4. Méthodologies de vérification et de validation

- **Analyse dynamique** : Cette approche nécessite l'exécution du programme. En exécutant le logiciel avec des cas de test spécifiques, les développeurs peuvent observer son comportement et identifier les défaillances. Cela inclut :
 - **Niveaux de test** : Tests unitaires, d'intégration et système. **Méthodes**
 - **avancées** : Analyse des mutations et exécution symbolique.
- **Analyse formelle** : La forme la plus rigoureuse de vérification et de validation, les méthodes formelles, utilisent des preuves mathématiques pour vérifier la correction d'un système. En utilisant des notations spécialisées (**comme la notation Z ou la méthode B**), les développeurs peuvent démontrer mathématiquement que la logique du logiciel est irréprochable.

Qu'est-ce que le test logiciel ?

Les tests logiciels constituent une étape cruciale du processus de développement logiciel. Il s'agit du processus d'évaluation d'un système ou d'une application logicielle afin de détecter les dysfonctionnements. **défauts, erreurs, ou insectes** et en vérifiant qu'il répond à ses exigences et fonctionne correctement.

Qu'est-ce que le test logiciel ?

Vous trouverez ci-dessous différentes définitions des tests :

«**Essai**» Il s'agit du processus consistant à mettre à l'épreuve ou à évaluer un système ou un composant de système par des moyens manuels ou automatisés afin de vérifier qu'il satisfait aux exigences spécifiées. »
[IEEE 83a]

«**Tests logiciels**» Il s'agit du processus d'exécution d'un programme ou d'un système dans le but de trouver des erreurs. »
[Myers]

Cela englobe toute activité visant à évaluer un attribut ou une capacité d'un programme ou d'un système et à déterminer s'il atteint les résultats escomptés.
[Hetzel]

Quelques remarques sur les définitions des tests :

Tests logiciels :

- **Vise à réduire le risque d'apparition d'anomalies par des moyens manuels et informatisés.**
- **Cela fait partie du processus de**
- **développement. N'a PAS pour but :**
 - **Corriger le défaut détecté (débogage). Prouver le**
 - **bon déroulement de l'exécution du programme.**

Procédure de test : Un échantillon de données d'entrée et de conditions environnementales est appliqué à tout ou partie du système informatique, et nous vérifions si le résultat obtenu est conforme au résultat attendu. Dans le cas contraire, cela signifie que le système informatique testé présente une anomalie de fonctionnement. (Tests logiciels) (également appelée vérification dynamique.)

Principes fondamentaux des tests logiciels

Pourquoi devrions-nous effectuer des tests ?

L'argumentaire technique :

Les développeurs compétents ne le sont pas **infaillible**.

Les implications des exigences ne sont pas toujours **prévisible**.

Le comportement d'un système n'est pas **nécessairement prévisible** à partir de ses composants.

Les langages, les bases de données, les interfaces utilisateur et les systèmes d'exploitation contiennent des bogues susceptibles d'entraîner des défaillances d'applications.

Principes fondamentaux des tests logiciels

Pourquoi devrions-nous effectuer des tests ?

L'argumentaire commercial :

- 1 Si vous ne trouvez pas les bugs, vos clients ou utilisateurs les trouveront. .**
- 2 Le débogage post-lancement est la forme de développement la plus coûteuse.**
- 3. Un logiciel défectueux nuit aux opérations, aux ventes et à la réputation.**

Pourquoi devrions-nous effectuer

des tests ? Le cas professionnel :

- 1. La conception des cas de test est une tâche à la fois stimulante et gratifiante.**
- 2. Des tests de qualité permettent d'avoir confiance en son travail.**
- 3. Des tests systématiques vous permettent d'être plus efficace.**
- 4 Votre crédibilité s'accroît et vous êtes fier de vos efforts.**

Principes fondamentaux des tests logiciels

Pourquoi devrions-nous effectuer des tests ?

Quelques chiffres clés :

Environ 70 % du temps de développement a été consacré à la phase de test.

Entre 30 et 50 % du budget total du projet ont été consacrés à la seule phase de test.

Les tests logiciels représentent environ 40 % de l'effort total.



Principes des tests

- 1. Les tests doivent être basés sur les exigences des utilisateurs : ceci afin de déceler tout défaut susceptible d'empêcher le programme ou le système de répondre aux exigences du client.**
- 2. Le temps et les ressources consacrés aux tests sont limités : évitez les tests redondants.**
- 3. Les tests exhaustifs sont impossibles : comme l'a déclaré Myer, il est impossible de tout tester en raison de l'immense espace de données et du grand nombre de chemins que peut emprunter un flux de programme.**

Principes des tests

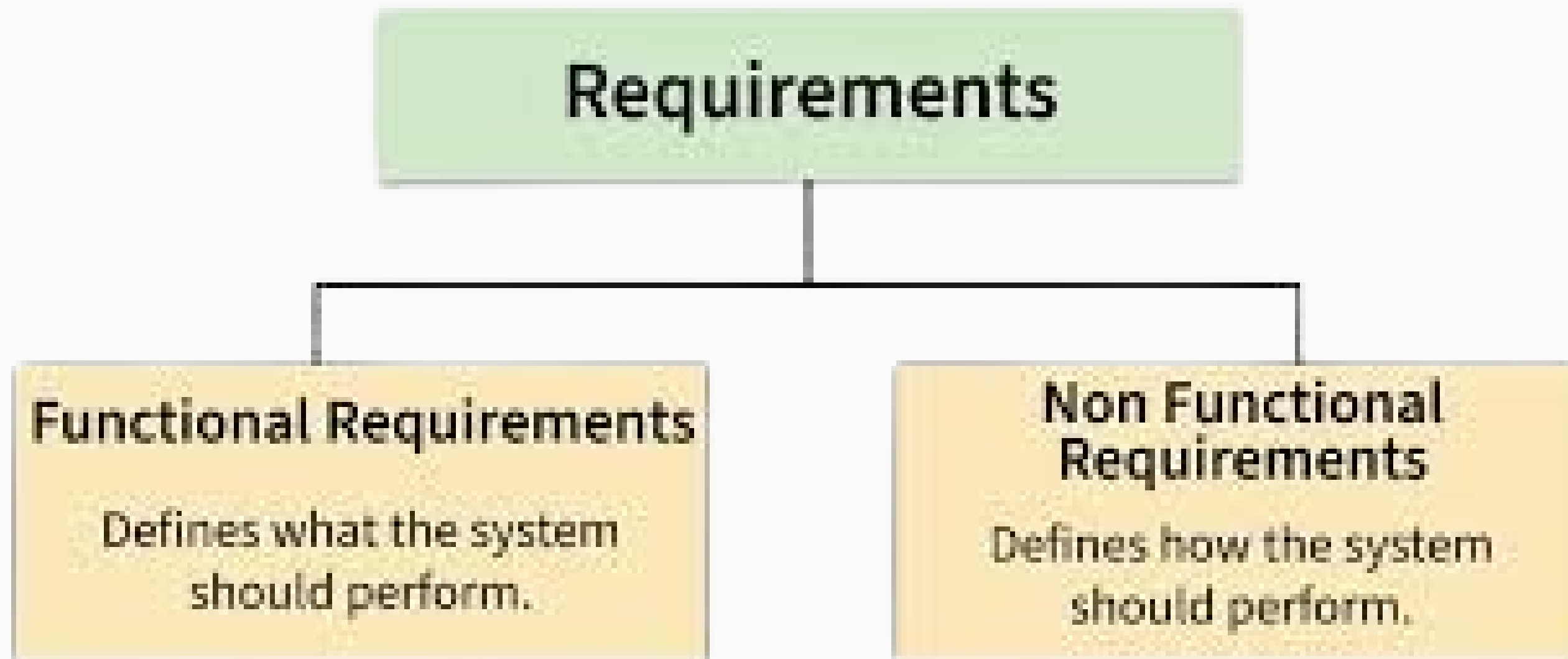
- 4. La planification des tests doit être effectuée tôt : en effet, la planification des tests peut commencer indépendamment du codage et dès que les exigences du client sont définies.**
- 5. Les tests doivent commencer « à petite échelle » et progresser vers des tests « à grande échelle » : les plus petites unités de programmation (ou modules) doivent être testées en premier, puis étendues à d'autres parties du système.**
- 6. Les tests doivent être effectués par un tiers indépendant.**
- 7. Tous les tests doivent être traçables par rapport aux exigences du client.**

Principes des tests

- 9. Confiez les tests aux personnes les plus compétentes. Évitez les programmeurs.**
- 10. Les tests doivent être conçus pour mettre en évidence les défauts logiciels et non leur absence.**
- 11. Préparer des rapports de test, comprenant des cas de test et des résultats de test, pour résumer les résultats des tests.**
- 12. Une planification préalable des tests est indispensable et doit être mise à jour rapidement.**

Exigences fonctionnelles et non fonctionnelles

En informatique, les exigences logicielles décrivent ce que le système doit faire et/ou ne pas faire, ainsi que les contraintes qui encadrent son fonctionnement. Il en existe deux types.



Révision des concepts importants

Exigences fonctionnelles et non fonctionnelles

Exigences fonctionnelles (Le « **Quoi** ») Ces éléments définissent les caractéristiques et les comportements spécifiques du système. Ils décrivent ce que fait le système.

Exemple: « Le système doit permettre aux utilisateurs d'ajouter des articles à leur panier. »

exigences non fonctionnelles (Le « **Comment** ») Ces éléments définissent les attributs, les contraintes et les objectifs de performance des tests. Ils décrivent le fonctionnement du système.

Exemple: « Le panier d'achat doit se mettre à jour dans les 500 millisecondes suivant l'ajout d'un article. »

Révision des concepts importants

Exigences fonctionnelles et non fonctionnelles

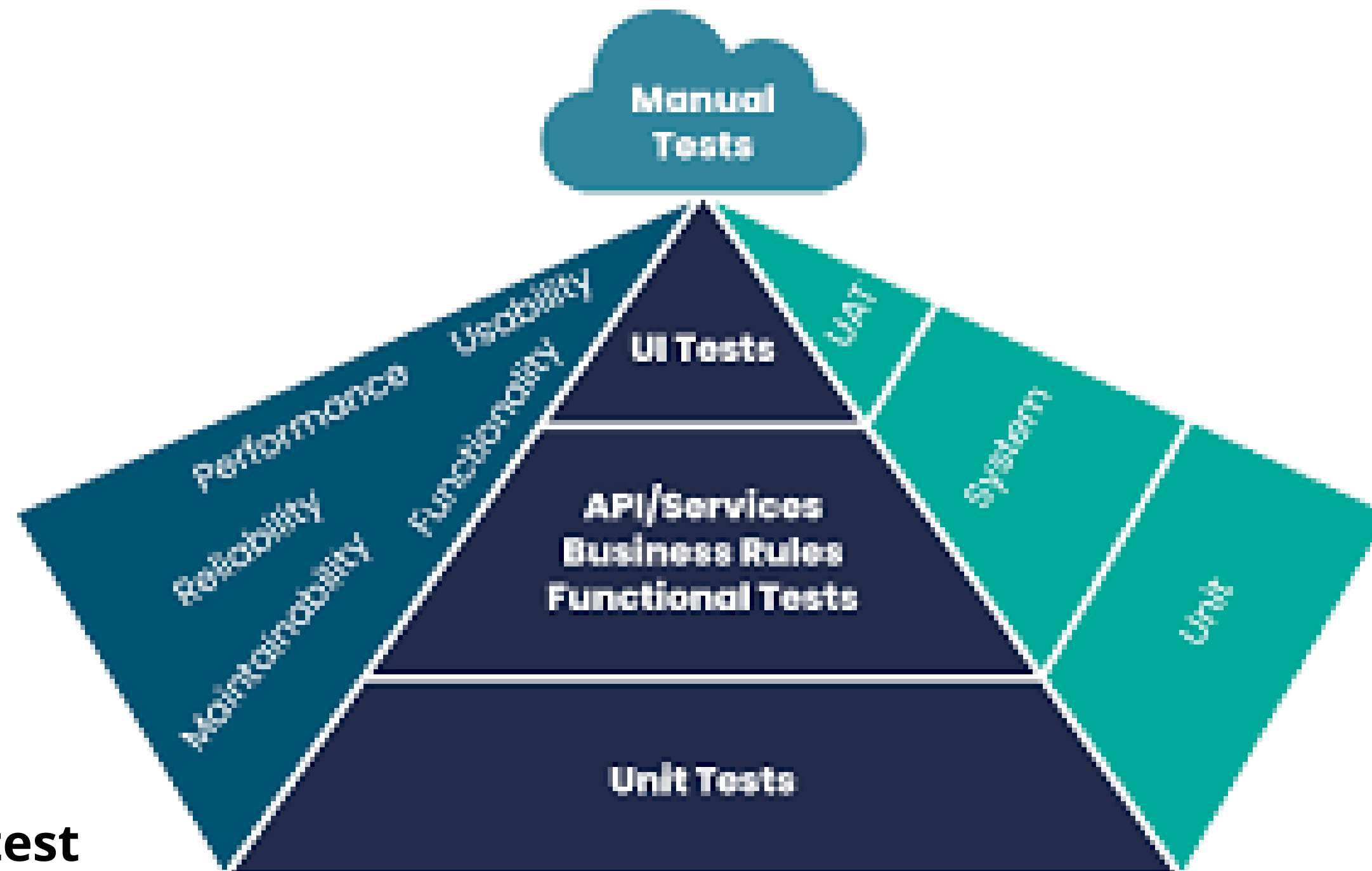
Fonctionnalité	Exigences fonctionnelles	exigences non fonctionnelles
Se concentrer	Tâches et fonctionnalités de l'utilisateur.	Propriétés et contraintes du système.
Question	Que fait ce système ?	Comment le système fonctionne-t-il ?
Exigence	Nécessaire au fonctionnement du système.	Obligatoire pour que le système soit efficace.
Mesures	Généralement binaire (Réussite/Échec).	Généralement quantitatif (métriques/échelles).
Impact de la défaillance	Le système est incomplet.	Le système est peu fiable ou frustrant.

Exigences fonctionnelles et tests fonctionnels

Ce type de test est réalisé pour vérifier que le logiciel fonctionne comme prévu et répond aux exigences de l'utilisateur. L'accent est mis principalement sur la fonctionnalité du logiciel et il comprend différents niveaux de tests, tels que : tests unitaires, tests d'intégration, tests système, et tests d'acceptation

Révision des concepts importants

Exigences fonctionnelles et tests fonctionnels



Pyramide de test

Exigences non fonctionnelles et tests non fonctionnels

Ce type de test porte sur les aspects non fonctionnels du logiciel, tels que ses performances, sa sécurité et son ergonomie. L'objectif est de garantir que le logiciel réponde à ses exigences non fonctionnelles et offre une expérience utilisateur positive.

Tests de performance Cela implique de tester les performances du logiciel dans diverses conditions, telles que la charge, la contrainte et l'évolutivité.

Tests de sécurité Cela implique de tester les fonctionnalités de sécurité du logiciel afin de s'assurer qu'il est protégé contre différents types d'attaques et de vulnérabilités.

Révision des concepts importants

Exigences non fonctionnelles et tests non fonctionnels

Tests d'utilisabilité Cela implique de tester l'interface utilisateur et l'expérience utilisateur du logiciel afin de s'assurer qu'il est convivial et facile à utiliser.

Tests de compatibilité Il s'agit de tester si le système peut fonctionner avec différents logiciels, matériels et systèmes d'exploitation.

Tests de mise à l'échelle teste si le système peut gérer des volumes croissants de données et d'utilisateurs.

Défis et limites des tests

1. L'« absence d'erreurs »

Erreur de raisonnement : les tests logiciels peuvent démontrer la présence de défauts, mais ils ne peuvent jamais prouver leur absence totale. Quelle que soit la rigueur du processus, on ne peut jamais garantir que le dernier bogue a été identifié et éliminé.

2. L'impossibilité des tests exhaustifs

Tester toutes les combinaisons possibles d'entrées et de chemins logiques est impossible, tant sur le plan du calcul que sur celui de la pratique. Le domaine des entrées est souvent trop vaste et le nombre de chemins d'exécution au sein d'un programme est trop complexe pour être couvert intégralement. Le principal défi des tests consiste à obtenir une couverture maximale avec un nombre minimal de cas de test.

3. Exigences de haute fiabilité vs. contraintes pratiques

Certains systèmes critiques, tels que les logiciels aéronautiques, exigent une fiabilité extrême (par exemple, un taux de défaillance inférieur à 10 %).⁻¹¹ par heure de vol). Si les composants aéronautiques sont conçus selon des normes aussi strictes, de nombreux systèmes du quotidien (comme les automobiles) présentent encore de multiples points de défaillance. Malgré les progrès technologiques réalisés dans la conception web, les applications mobiles et l'intelligence artificielle, une technologie logicielle « parfaite » reste un rêve inaccessible, et il est peu probable qu'elle voie le jour.

Défis et limites des tests

4. Ambiguïté des exigences

Les tests sont souvent entravés par des spécifications incomplètes, informelles ou en constante évolution. Lorsque les exigences des utilisateurs sont mal définies ou changent au cours du développement, la validation du logiciel par rapport à son usage prévu devient un exercice complexe.

5. Évaluation des mesures et des tests

Quantifier l'efficacité des tests et l'impact global des tests logiciels demeure un défi majeur. Il est difficile de déterminer précisément la quantité de tests nécessaires ou d'évaluer leur impact exact sur la fiabilité du produit final.

6. La nécessité d'une approche multifacettes

Aucune méthode de test n'étant infallible, il convient de considérer les différentes techniques comme complémentaires. Seule une combinaison stratégique de méthodologies de test permet d'espérer détecter la majorité des erreurs et garantir un système robuste.

Module II : Catégories, méthodologies et...

Cycle de vie (STLC)

Catégories de tests logiciels

Les tests logiciels peuvent être globalement classés en deux catégories d'exécution. modes :

Tests manuels

Le processus dans lequel un testeur endosse le rôle d'un utilisateur final pour interagir manuellement avec le logiciel sans scripts ni outils.

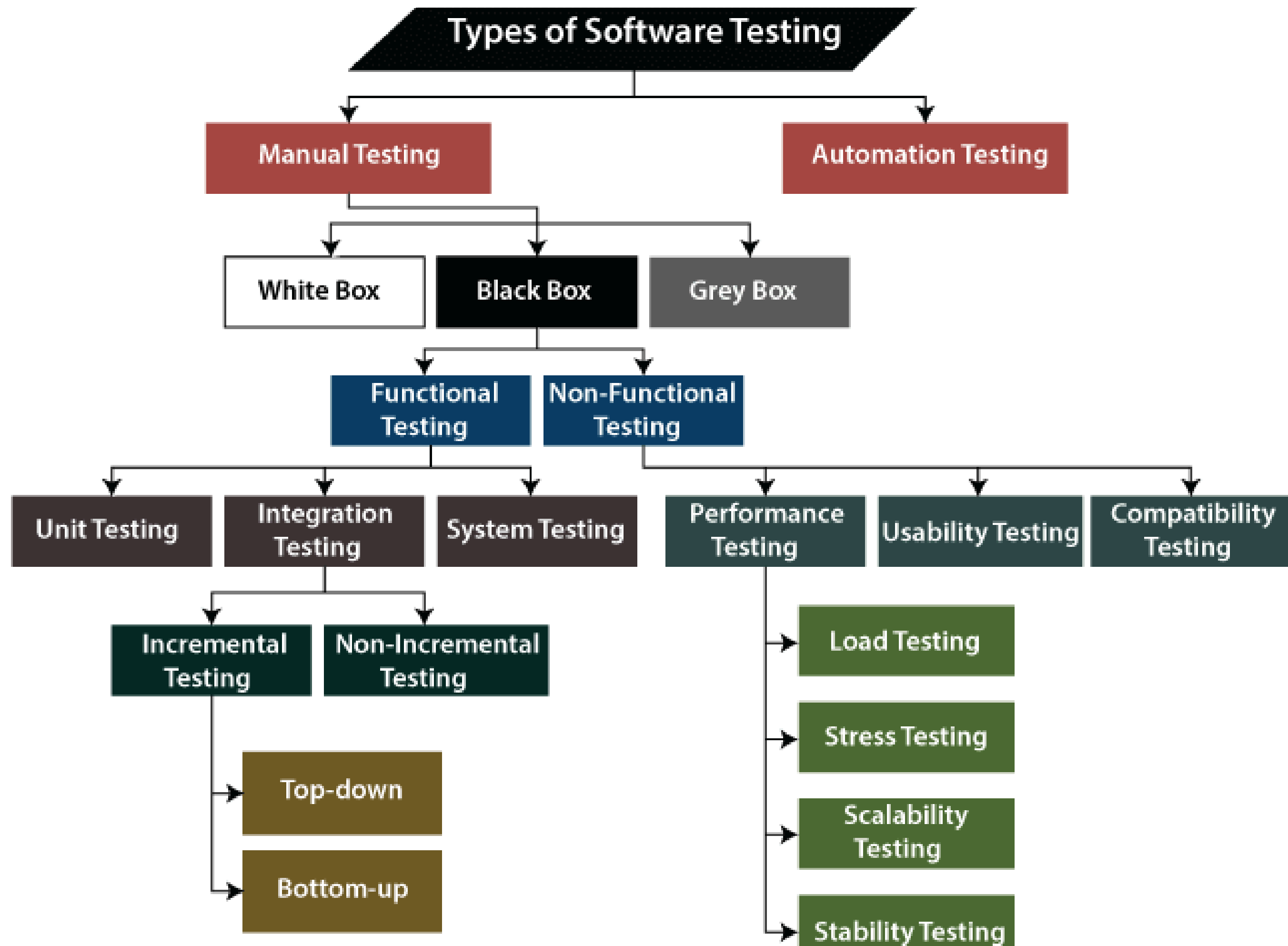
L'objectif est d'identifier les bugs et les comportements inattendus.

Il comprend des étapes telles que les tests unitaires, d'intégration, système et d'acceptation utilisateur (UAT), utilisant souvent des tests exploratoires pour découvrir les problèmes non enregistrés.

Catégories de tests logiciels

Tests d'automatisation

- **Les tests automatisés, également appelés « automatisation des tests », sont lorsque le testeur écrit des scripts de test et utilise un autre logiciel pour tester le logiciel.**
- **Ce processus consiste à automatiser une procédure manuelle. Les tests automatisés permettent de réexécuter rapidement et de manière répétée les scénarios de test auparavant réalisés manuellement.**
- **Outre les tests de régression, les tests automatisés permettent également de tester l'application du point de vue de la charge, des performances et de la résistance. Ils augmentent la couverture des tests, améliorent la précision et permettent de gagner du temps et de l'argent par rapport aux tests manuels.**



Catégories de tests logiciels

Tests d'automatisation

Que faut-il automatiser ?

- Il n'est pas possible d'automatiser l'intégralité du logiciel ; cependant, les zones où l'utilisateur peut effectuer des transactions, telles que les formulaires de connexion ou d'inscription, etc., et **toute zone où un grand nombre d'utilisateurs peuvent accéder simultanément au logiciel**
- De plus, tous les éléments d'interface graphique, les connexions aux bases de données, les validations de champs, etc.
- **peut être testé efficacement en automatisant le processus manuel.**

Catégories de tests logiciels

Tests d'automatisation

Quand automatiser ?

L'automatisation des tests doit être utilisée en tenant compte des éléments suivants pour le logiciel :

- Projets de grande envergure et essentiels.
- Projets nécessitant des tests fréquents dans les mêmes domaines. Exigences peu susceptibles de changer.
- Évaluation de la charge et des performances de l'application avec de nombreux utilisateurs virtuels.
- Logiciel stable du point de vue des tests manuels.
- Disponibilité du temps.

Catégories de tests logiciels

Tests d'automatisation

Le processus à automatiser ?

L'automatisation s'effectue à l'aide d'un langage informatique dédié, tel que VB Script, et d'une application logicielle automatisée. De nombreux outils permettent d'écrire des scripts d'automatisation, comme indiqué précédemment.

Les processus qui peuvent être utilisés pour réaliser et automatiser les tests :

- Identification des zones d'un logiciel à automatiser. Sélection
- des outils appropriés pour l'automatisation des tests.
- Rédaction des scripts de test.
- Développement de suites de
- tests. Exécution de scripts.
- Créer des rapports de résultats.
- Identifier tout bug potentiel ou problème de performance.

Catégories de tests logiciels

Tests d'automatisation

Quelques outils d'automatisation

Voici les outils qui peuvent être utilisés pour les tests d'automatisation :

HP Quick Test Professional, Selenium, IBM Rational Functional Tester, SilkTest, TestComplete, Testing Anywhere, WinRunner, LoadRunner, Visual Studio Test Professional, WATIR

Catégories de tests logiciels

Conception et gestion des cas de test

La rédaction de cas de test de haute qualité est la responsabilité principale d'un **analyste de tests**. Cette activité est cruciale pour améliorer les tests logiciels et garantir une validation complète.

Principes fondamentaux des cas de test

Un cas de test (TC) Un test de validation est un ensemble de données d'entrée spécifiques, de conditions d'exécution et de résultats attendus, conçu pour valider un objectif ou une fonctionnalité particulière. Il détermine si une fonction spécifique de l'application fonctionne correctement.

- **Suite de tests** : Un regroupement logique ou une collection d'éléments apparentés **cas de test**.
- **Objectif principal** : Pour valider le **couverture des tests** d'une application, en veillant à ce que tous les aspects fonctionnels et non fonctionnels soient vérifiés.

Conception et gestion des cas de test

Le rôle de l'analyste de tests

Pour être efficace, un analyste de tests doit répondre à trois questions fondamentales :

1. **Quelle est l'activité principale ?** Concevoir et exécuter des tests qui mettent à l'épreuve les limites du logiciel.
2. **La connaissance du domaine est-elle nécessaire ?** Oui. Un analyste ne peut pas rédiger des cas de test efficaces sans une compréhension approfondie du **domaine d'application** (le contexte commercial dans lequel le logiciel fonctionne).
3. **Pourquoi les écrire ?** Fournir une méthode reproductible, documentée et mesurable pour confirmer que le logiciel répond à ses exigences.

Conception et gestion des cas de test Sources

d'information et critères de sélection

Les cas de test ne sont pas créés ex nihilo ; ils sont dérivés de :

- **Documents de spécification des exigences (SRS) :** Pour assurer un alignement fonctionnel.
- **Documents de conception :** Pour comprendre la logique structurelle.

Critères de sélection des cas de test :

1. **Établir un cadre d'évaluation d'un ensemble spécifique de valeurs d'entrée.**
2. **Générer des ensembles de données d'entrée qui satisfont à ces critères établis.**

Catégories de tests logiciels

Conception et gestion des cas de test Étapes

systematiques pour la rédaction des cas de test

Le processus de création suit une méthodologie structurée :

1. **Génération de scénarios** :Élaboration de diagrammes de cas d'utilisation et création d'une matrice de scénarios de cas d'utilisation.
2. **Identification des variables** :Identifier les variables au sein d'un cas d'utilisation et déterminer leurs différents états d'entrée.
3. **Conception matricielle** :Construction d'une matrice de cas de test pour associer les scénarios aux variables.
4. **Affectation des données** :Attribution de valeurs réelles et concrètes aux variables identifiées pour exécution.

Catégories de tests logiciels

Conception et gestion des cas de test

Format de cas de test (modèle) 1

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status

Conception et gestion des cas de test

Meilleures pratiques en matière de formatage des cas de test

- **Tests défensifs** : Les cas de test doivent être « défensifs », c'est-à-dire qu'ils doivent couvrir à la fois les conditions normales (entrées valides) et les conditions anormales (entrées invalides/inattendues).
- **Couverture universelle** : Pour toute application, les cas de test doivent aborder les tests de performance, de sécurité et de régression.
- **Traçabilité** : Chaque cas de test DOIT être lié à une exigence spécifique pour justifier son existence et garantir qu'aucune exigence ne soit négligée.

Catégories de tests logiciels

La matrice de traçabilité des exigences (RTM)

Le RTM est un outil stratégique (généralement un tableau) utilisé tout au long du cycle de vie des tests logiciels (STLC) pour cartographier et suivre les exigences des utilisateurs par rapport à leurs cas de test correspondants.

Types de traçabilité

1. **Traçabilité en amont** : Il associe les exigences aux cas de test. Cela garantit que le projet progresse dans la bonne direction et que chaque exigence est testée de manière approfondie.
2. **Traçabilité ascendante (inverse)** : Il permet de relier les cas de test aux exigences. Il empêche les dérives de périmètre en garantissant qu'aucun code ou fonctionnalité inutile (non demandé par le client) n'a été ajouté.
3. **Traçabilité bidirectionnelle** : Une combinaison des deux. Elle assure une couverture totale et aide à analyser comment un défaut dans un produit affecte des exigences spécifiques et vice versa.

Catégories de tests logiciels

La matrice de traçabilité des exigences (RTM)

Principaux avantages du RTM

- **Conformité:** Garantit que le logiciel est développé en stricte conformité avec les spécifications du client.
- **Analyse des causes profondes :** Permet d'identifier l'origine des bogues. **Cohérence:** Met en évidence les exigences manquantes ou les incohérences dans la documentation.
- **État d'exécution :** Fournit une vue d'ensemble claire de l'avancement des tests et de l'état des anomalies, axée sur les objectifs commerciaux.
- **Analyse d'impact :** Aide l'équipe d'assurance qualité à estimer l'effort nécessaire lorsque les exigences changent et que les cas de test doivent être retravaillés.

Catégories de tests logiciels

Requirements Traceability Matrix									
Project Name		<Project Name here>	Created On	3-Oct-11	Reviewed On	4-Oct-11			
Release No		<Project Release>	Created By	<Creator's Name>	Reviewed By	<Reviewer's Name>			
Version		<Doc version>							
ID	Requirement ID	Requirement Description	Status	Design Document	Code Module	TestCase ID	Test Case Name	User Manual	Tested On/ Verification
001	UC 1.0	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-001	CM-001	TC-001	ProjName_UCID_TestCase Name	Section 4.5	Pending
002	UC 1.1	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-002	CM-002	TC-002 TC-003	N.A.	Section 4.6	Verified
003	UC 1.2	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-003	CM-003	TC-004 TC-006 TC-007 TC-008		Section 5.7	Verified
004	UC 1.3	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-004	CM-004			Section 6.8	In-progress
005	UC 1.4	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-005	CM-005			Section 7.9	Not Verified
006	UC 1.5	Testing Requirement Description here. It should not be more than 2-3 lines	TBD	DM-006	CM-006			Section 4.10	Verified
007	UC 1.6	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.11	Not Verified
008	UC 1.7	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.12	Pending
009	UC 1.8	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.13	Not Verified
010	UC 1.9	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.14	Pending

Le cycle de vie des tests logiciels (STLC)

Le **STLC** se compose de six phases systématiques, chacune avec des caractéristiques spécifiques **critères d'entrée** (prérequis) et **critères de sortie** (conditions d'obtention du diplôme) :

1. **Analyse des besoins** : Étude des exigences fonctionnelles et non fonctionnelles ; préparation de la matrice de traçabilité des exigences (RTM).
2. **Planification des tests** : Définir la stratégie, estimer les efforts et les coûts, et sélectionner les outils.
3. **Développement des cas de test** : Création et révision des cas de test, des scripts et des données de test.
4. **Configuration de l'environnement de test** : Configuration des conditions matérielles et logicielles ; réalisation de tests de fonctionnement préliminaires pour garantir la conformité de l'environnement.
5. **Exécution du test** : Exécution des tests, enregistrement des anomalies et nouveaux tests des correctifs.
6. **Clôture/Résultats du test** : Finalisation des rapports et évaluation de l'« Oracle de test » (le mécanisme permettant de déterminer si un test a réussi ou échoué).

Méthodologies de test : statiques vs dynamiques

Les tests peuvent être abordés selon deux perspectives techniques :

Tests statiques (boîte noire / fonctionnels)

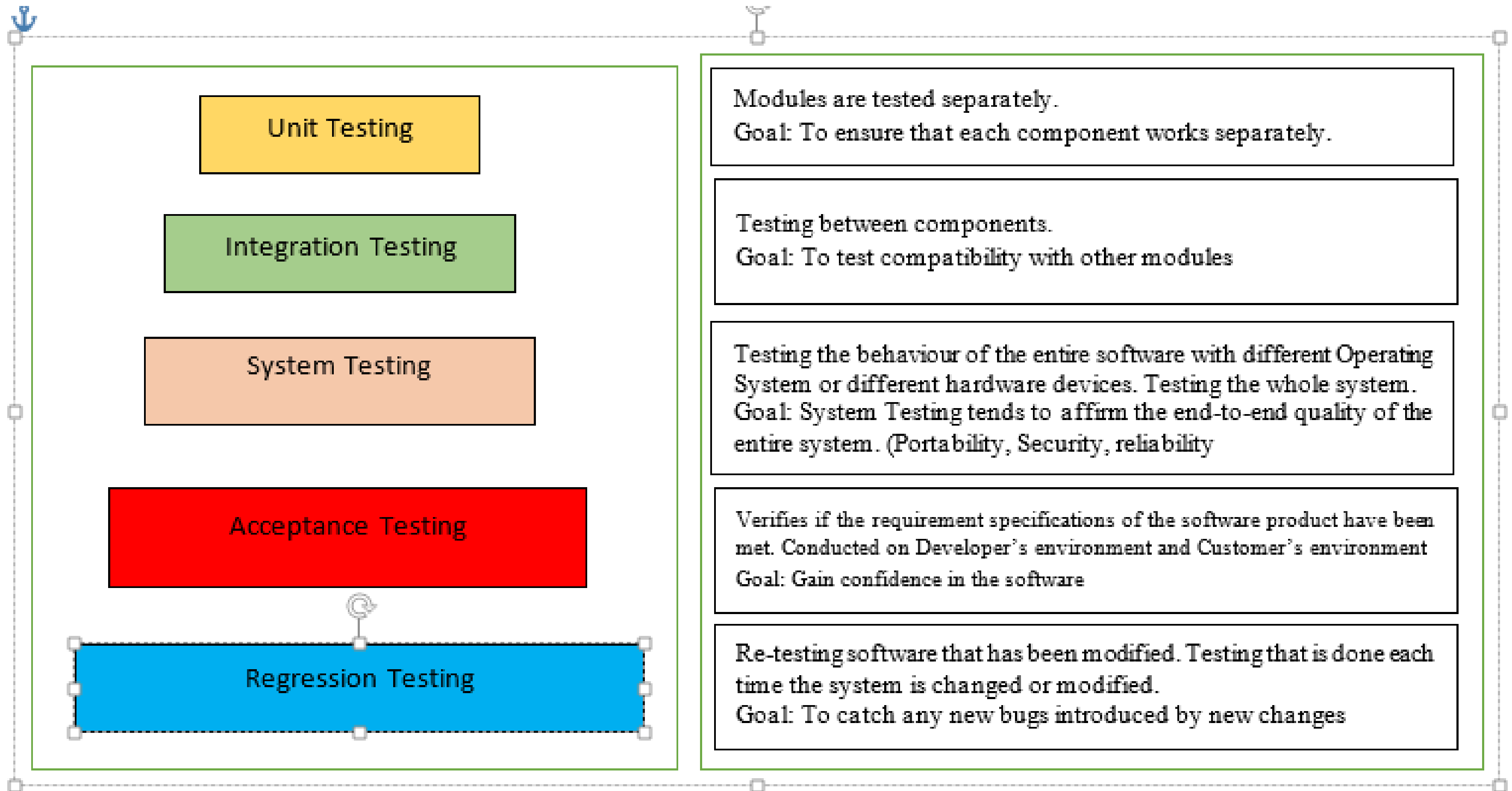
- **Concept : Tester sans exécuter le code.**
- **Point de vue : Il envisage le logiciel du point de vue de l'utilisateur, en se concentrant sur « ce que » fait le système en fonction des exigences.**
- **Techniques : Examens, inspections et analyses de documents.**

Méthodologies de test : statiques vs dynamiques

Tests dynamiques (boîte blanche / structurels)

- **Concept** : Tester le programme en l'exécutant avec des entrées réelles.
- **Objectif** : Analyse de la logique interne et de la structure du code (son fonctionnement).
- **Techniques de boîte blanche** :
 - **Tests de flux de contrôle** : Utilise la structure de contrôle du programme (graphes) pour développer des cas.
 - **Tests de flux de données** : se concentrent sur la manière dont les variables sont définies et utilisées.
- **Critères de couverture** : mesurent la mesure dans laquelle le code est testé (**couverture d'énoncé, de branche/décision, de chemin ou de prédicat**).

Tests dynamiques (boîte blanche / structurels)



Méthodologies de test : statiques vs dynamiques

Les limites des tests en boîte blanche :

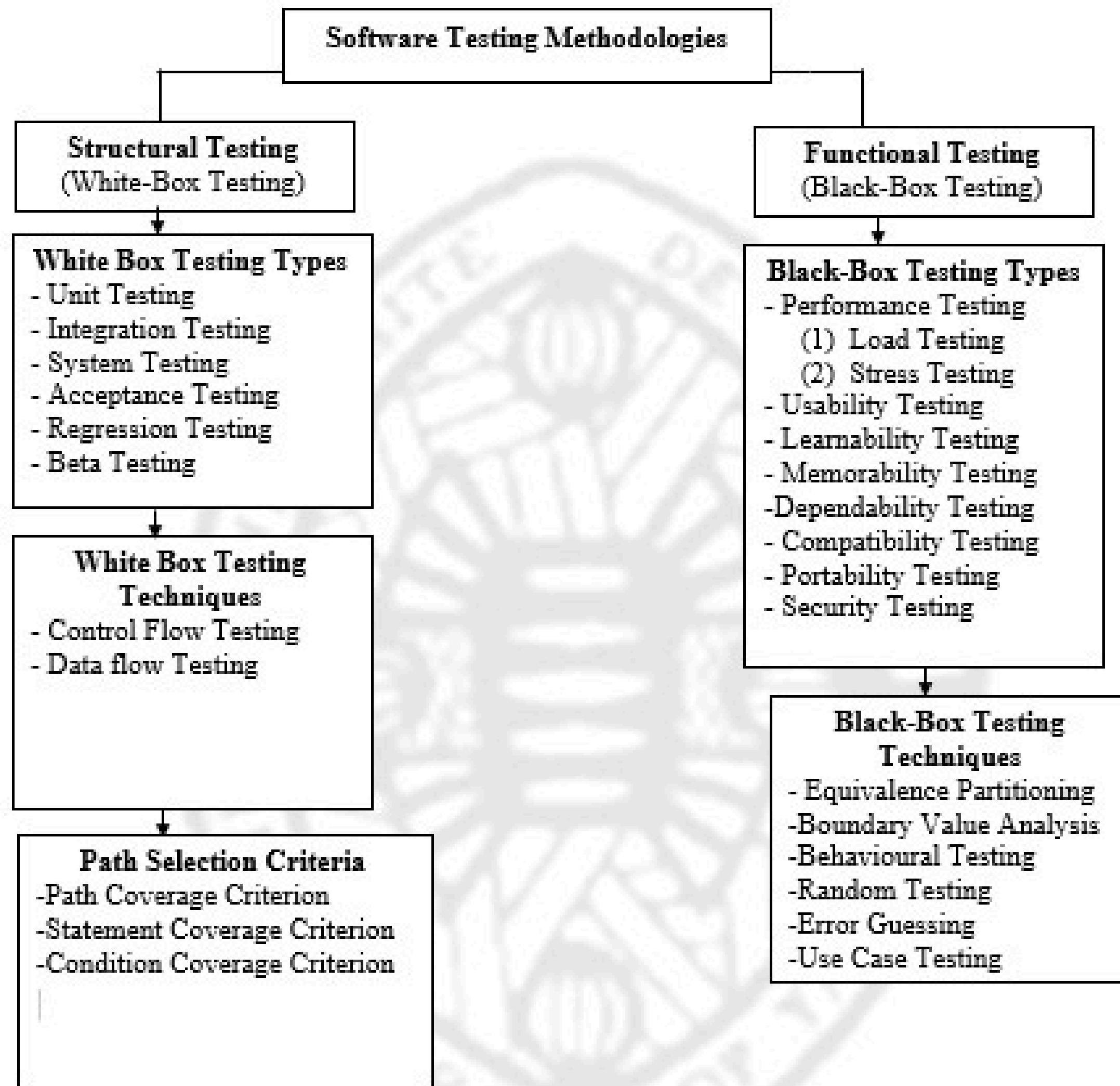
- Les développeurs effectuent souvent des tests dans le but de prouver que le code fonctionne plutôt que de prouver qu'il ne fonctionne pas.

Il existe de nombreux types d'erreurs que les tests en boîte blanche ne permettent pas de détecter :

- Problèmes de performance
- Problèmes d'utilisabilité

Tests de régression :

Cette opération est effectuée après toute modification, mise à jour ou maintenance du code afin de garantir que les nouveaux changements n'ont pas eu d'impact négatif sur les fonctionnalités stables existantes.



TECHNIQUES D'ESSAIS STRUCTURAUX

Dans les tests structurels, le logiciel est considéré comme une boîte blanche, et les cas de test sont déterminés à partir de l'implémentation du logiciel.

**Les techniques d'essais structureaux
comprennent : (1) Tests de flux de contrôle
(2) Tests de flux de données.**

Méthodologies de test : statiques vs dynamiques

Tests en boîte blanche/Tests structurels

- Tests basés sur le code source du programme
- Mesure dans laquelle le code (source) est exécuté, c'est-à-dire couvert
- Différents types de couverture (c'est-à-dire la mesure dans laquelle le code a été couvert)
 - Critère de couverture de chemin ; Énoncé du critère de
 - couverture ; Critère de couverture de condition/branche/
 - décision ; Critère de couverture de prédicat
 -

Méthodologies de test : statiques vs dynamiques

TECHNIQUES D'ESSAIS STRUCTURAUX

ESSAIS DE DÉBIT RÉGULATEUR

Les tests de flux de contrôle utilisent la **structure de contrôle** d'un programme permettant de développer les cas de test pour le programme.

Les cas de test sont développés pour **suffisamment couvrir** l'ensemble de la structure de contrôle du programme.

La structure de contrôle d'un programme peut être représentée par **graphe de flux de contrôle** du programme.

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

GRAPHIQUE DE FLUX DE CONTRÔLE (CFG)

- L'adéquation des cas de test est mesurée à l'aide d'une métrique appelée "**couverture**" (**Couverture** est une mesure de l'exhaustivité de l'ensemble des cas de test, c'est-à-dire la mesure dans laquelle le code a été couvert)
- Un critère de couverture de test mesure dans quelle mesure un ensemble de cas de test couvre un programme.
- **Un graphe de flux de contrôle** Un CFG est une représentation graphique d'une unité de programme. Autrement dit, un CFG est une représentation graphique de la structure de contrôle d'un programme.

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

GRAPHIQUE DE FLUX DE CONTRÔLE (CFG)

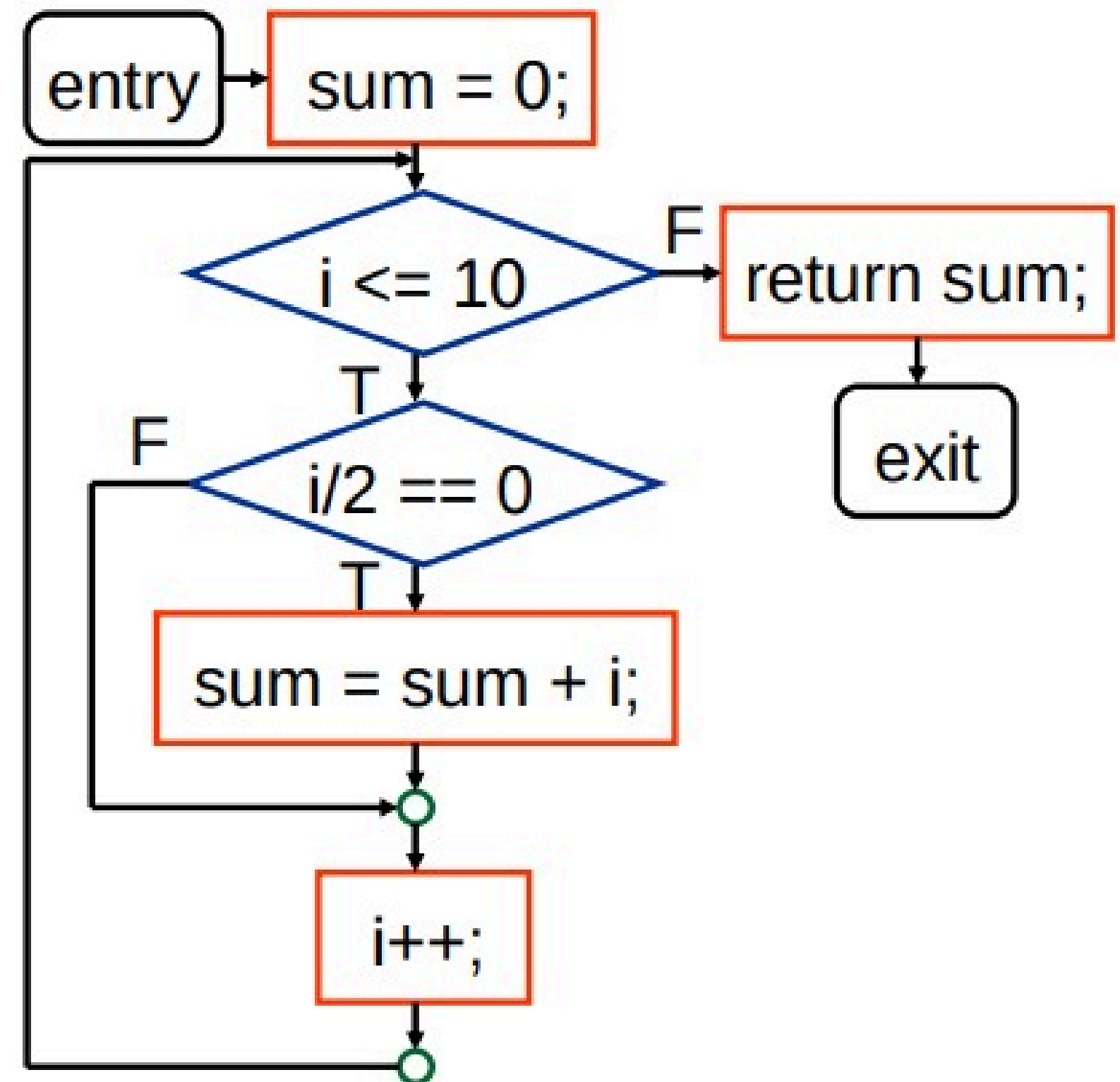
- Un graphe de flux de contrôle pour un programme est constitué d'un nœud pour chaque instruction du programme et d'arêtes représentant le flux de contrôle entre les instructions.
- Le graphe de flux de contrôle d'un programme est constitué d'un ensemble de nœuds N et d'un ensemble d'arêtes E .
- Chaque nœud représente un ensemble d'instructions de programme.
- Il existe un nœud d'entrée unique et un nœud de sortie unique.
- Il existe une arête provenant du nœud $n1$ vers le nœud $n2$ Si le contrôle peut découler de la dernière instruction dans $n1$ à la première déclaration dans $n2$

FLUX DE CONTRÔLE ESSAI

Control Flow Graph: An Example

```
int evensum(int i)
{
    int sum = 0;

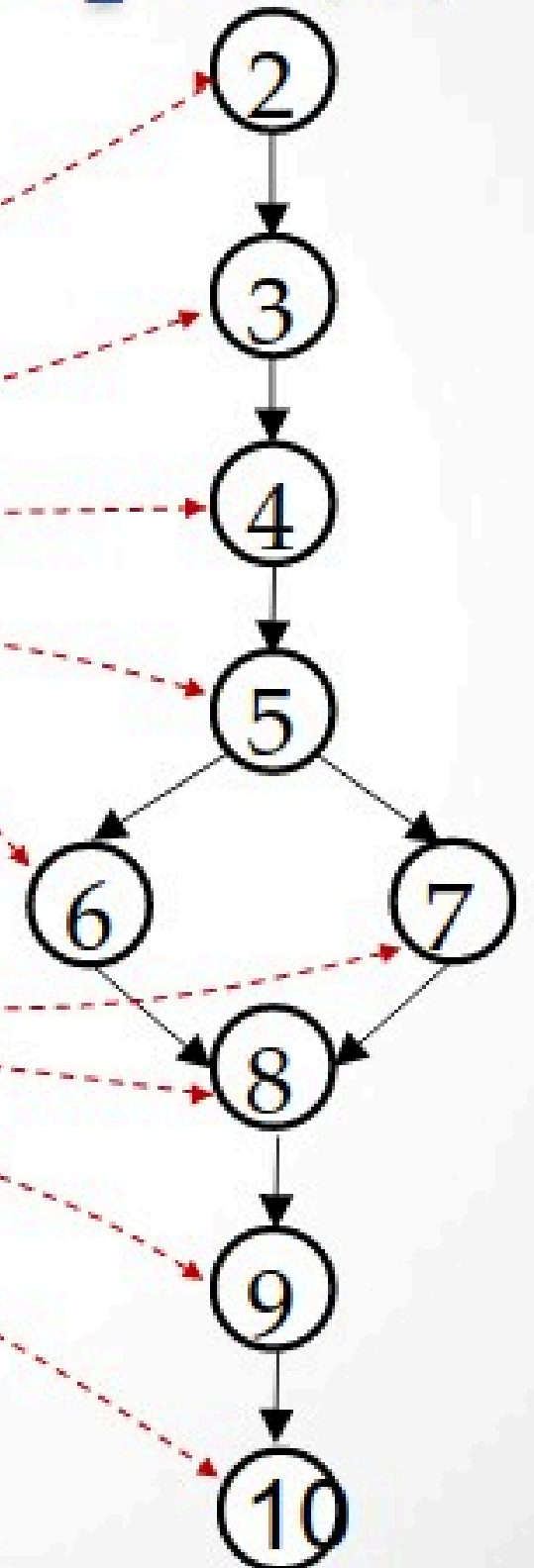
    while (i <= 10) {
        if (i/2 == 0)
            sum = sum + i;
        i++;
    }
    return sum;
}
```



ESSAIS DE DÉBIT RÉGULATEUR

Flow Graph Example(1)

1. Program 'Simple Subtraction'
2. Input(x,y)
3. Output (x)
4. Output(y)
5. If $x > y$ then Do
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output(z)
10. Output "End Program"

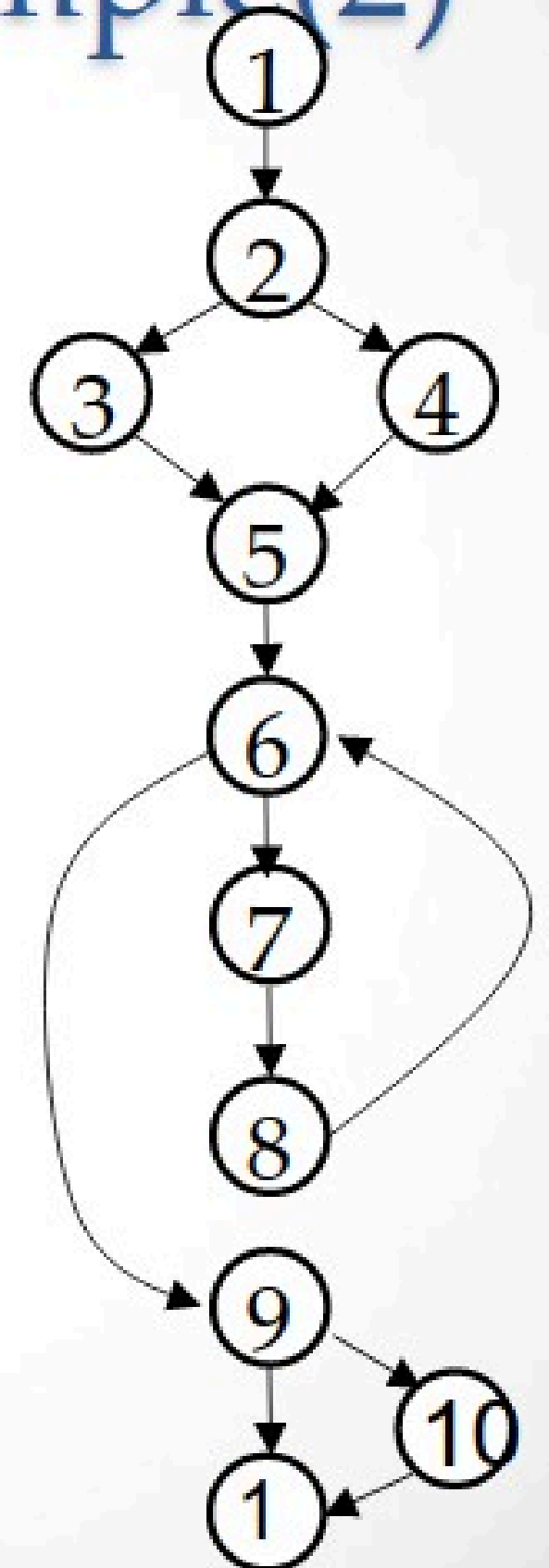


Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

Flow Graph Example(2)

```
1. scanf("%d %d",&x, &y);
2. If (y<0)
3.     pow = -y;
   else
4.     pow = y;
5. Z = 1.0
6. While (pow != 0) {
7.     z = z*x;
8.     pow = pow -1; }
9. If (y <0)
10. z = 1.0 /z;
11. printf("%f",z);
```



Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

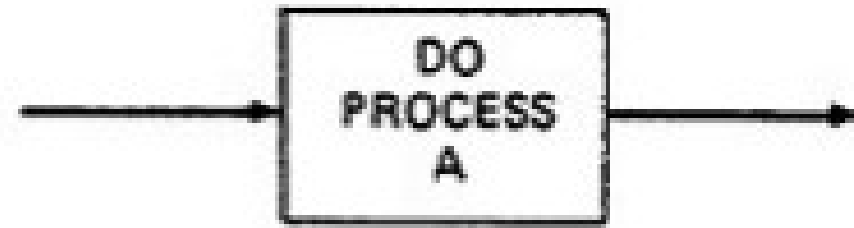
GRAPHIQUE DE FLUX DE CONTRÔLE (CFG) : Types d'éléments

Un graphe de flux contient quatre types d'éléments différents (nœuds).

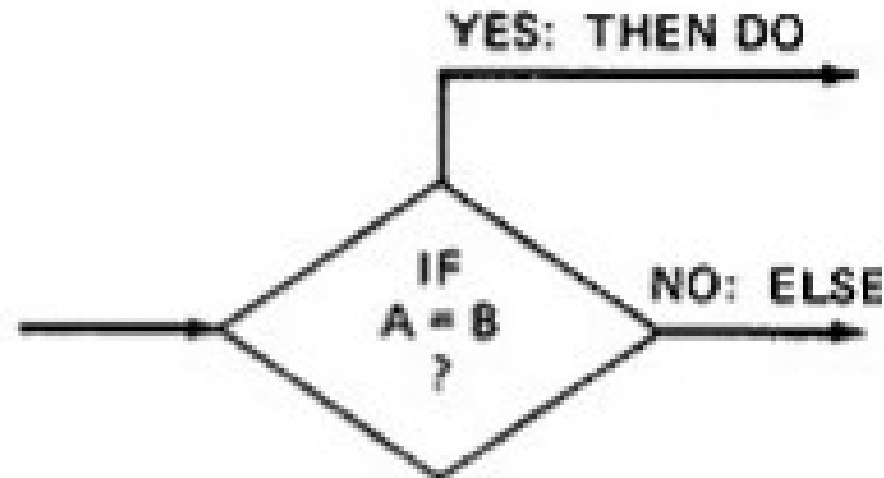
- Bloc de processus
- Décisions
- Jonctions
- Déclarations de cas

GRAPHIQUE DE FLUX DE CONTRÔLE (CFG) : Types d'éléments

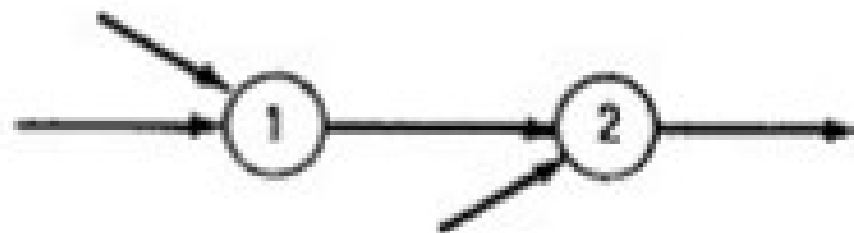
Processus



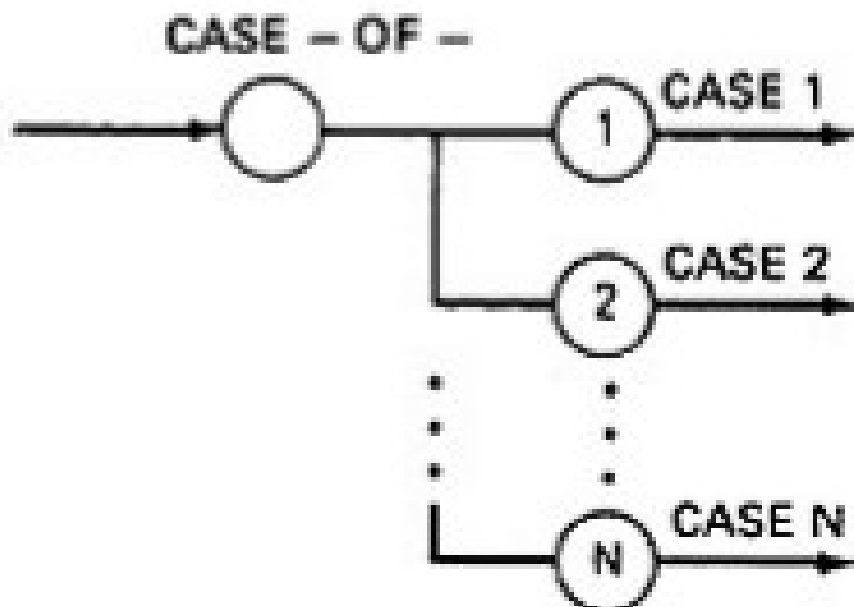
Décisions



Junctions



Case Statement

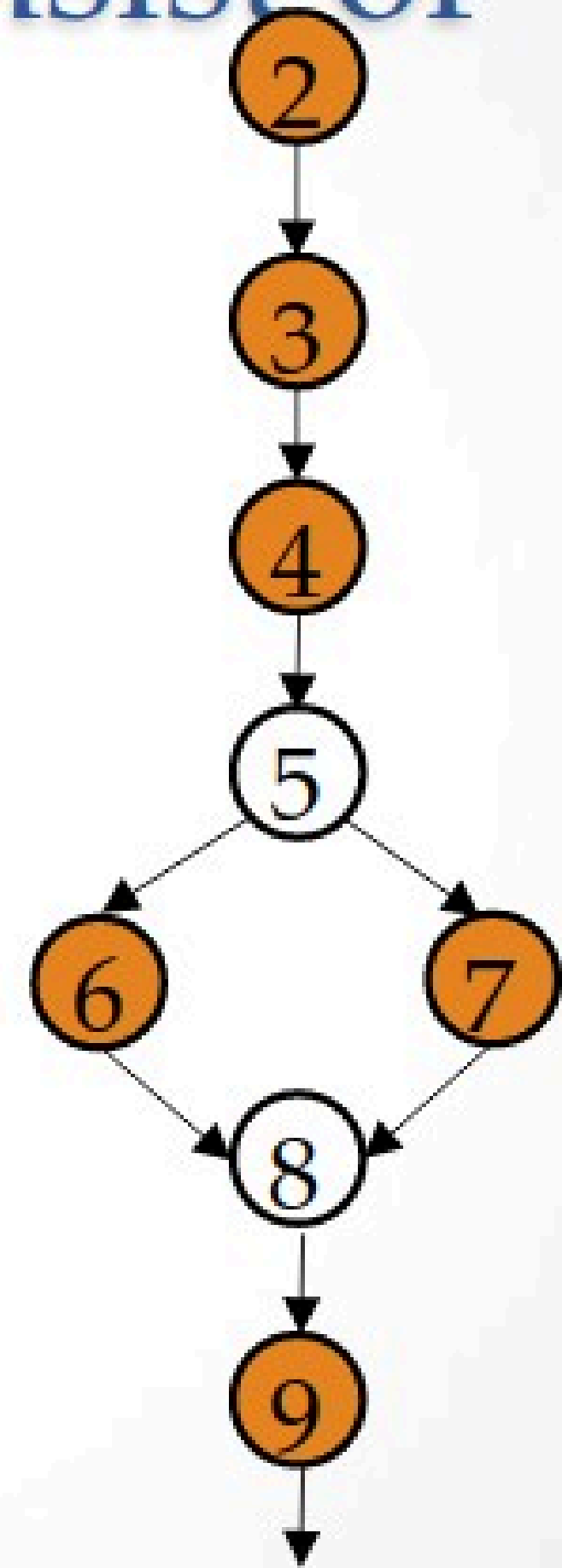


- **Un bloc de processus**(Lien de processus) est une séquence d'instructions de programme non interrompue par des décisions ou des jonctions.
- **Un nœud de décision** Un nœud de décision est un point d'exécution où le flux de contrôle peut diverger. Autrement dit, un nœud de décision contient une instruction conditionnelle qui crée au moins deux branches de contrôle (par exemple, des instructions if ou switch).
- **Un nœud de jonction** est un point du programme où le flux de contrôle peut converger.
- **Un nœud de cas** est une branche ou une décision à plusieurs voies.

Flow Graphs consist of

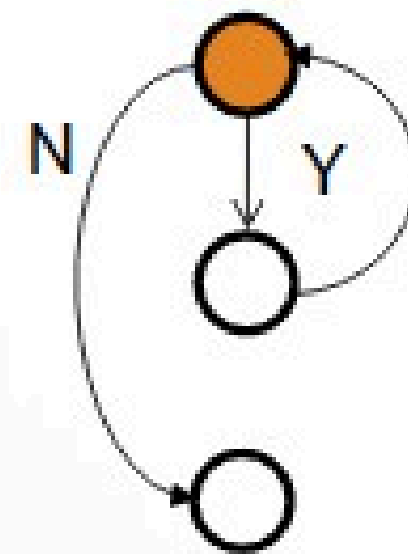
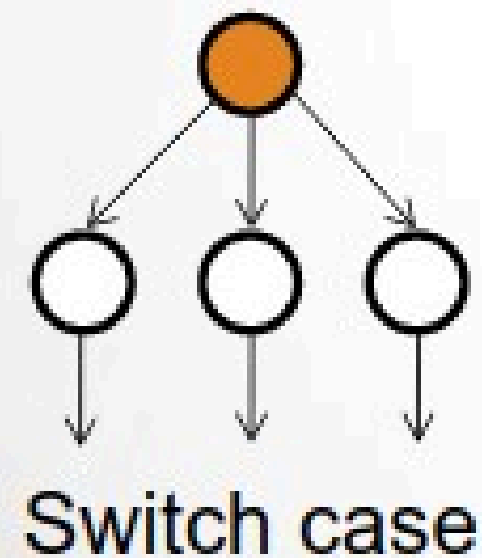
- A **process block** is a sequence of program statements uninterrupted by either decisions or junctions.
- (i.e. straight-line code)

Sequence

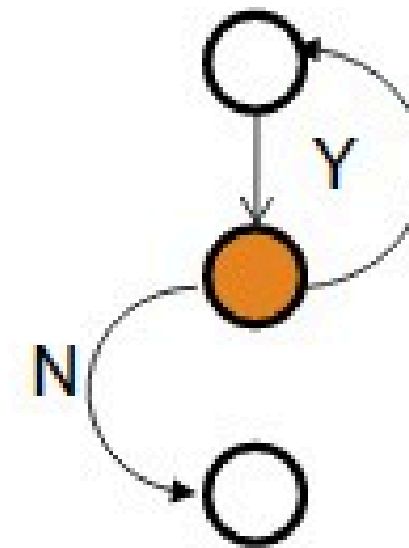


Flow Graphs consist of

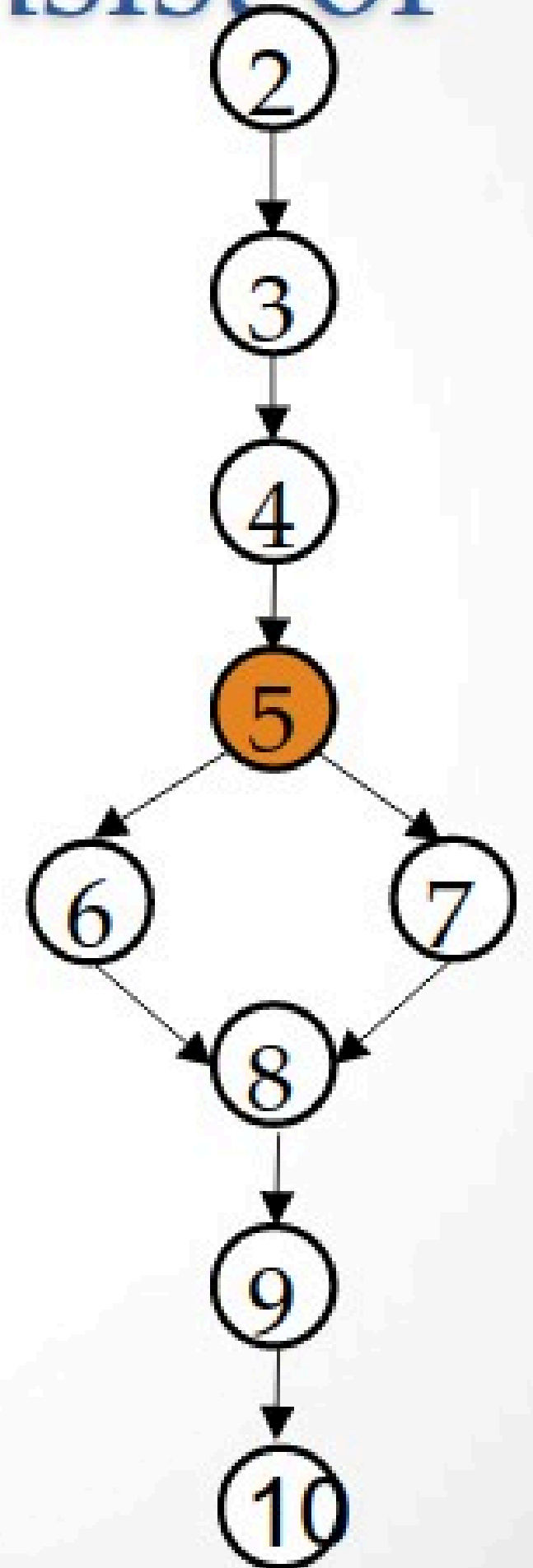
- A **decision** is a program point at which the control can diverge.
- (e.g., if and case statements)



While Loop

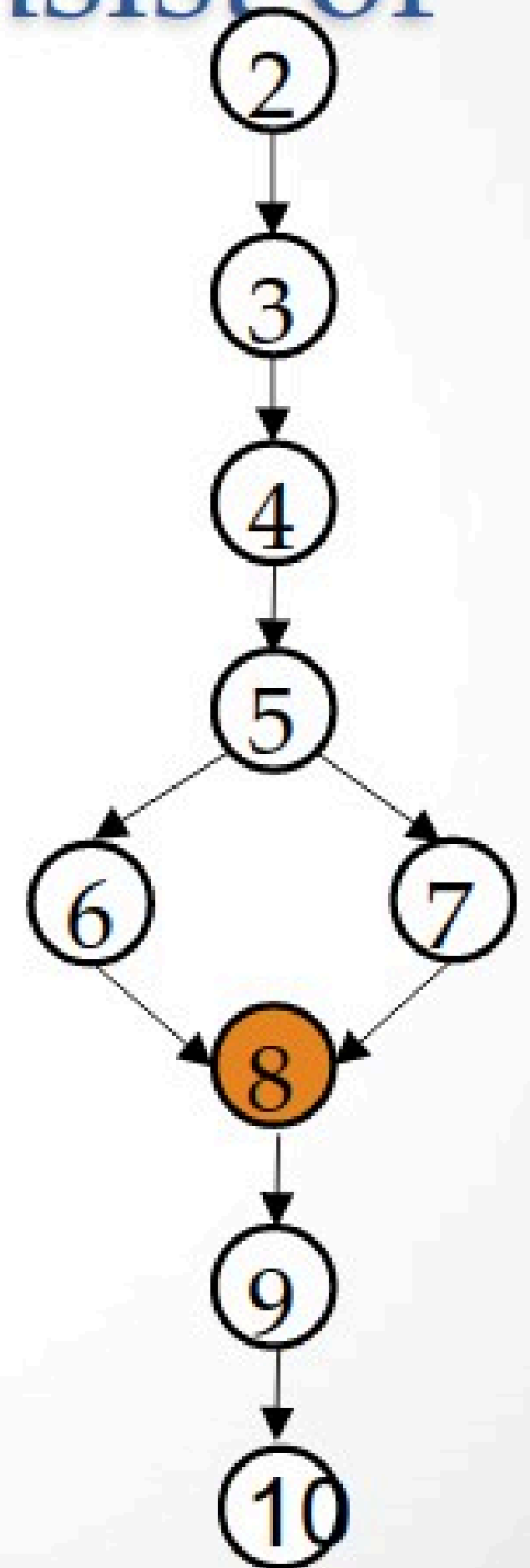
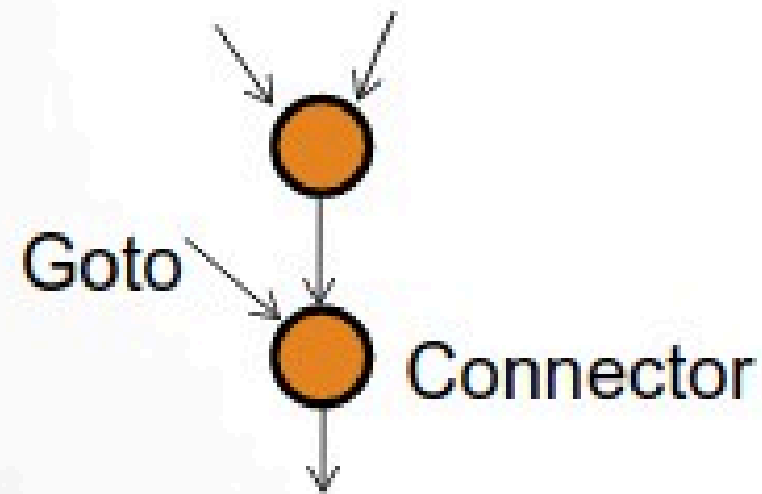


Until Loop



Flow Graphs consist of

- A **junction** is a program point where the control flow can merge.
- (e.g., end if , end loop, goto label)



Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

GRAPHIQUE DE FLUX DE CONTRÔLE (CFG) : Les limites des tests de flux de contrôle

Voici quelques-unes des limitations des tests de flux de contrôle :

- 1. Les tests de flux de contrôle ne peuvent pas détecter toutes les erreurs d'initialisation.**
- 2. Les erreurs de spécification des exigences ne sont pas détectées par les tests de flux de contrôle.**
- 3. Il est donc peu probable de trouver des chemins et des fonctionnalités manquants si le programme et le modèle sur lequel les tests sont basés sont réalisés par la même personne.**

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

Tests de chemin

Tests de chemin Il s'agit d'une méthode de test structurel qui consiste à utiliser le code source d'un programme pour tenter de trouver tous les chemins d'exécution possibles.

- L'idée est de pouvoir tester chaque chemin individuel à partir du code source de autant de manières que possible afin de maximiser la couverture de chaque cas de test.
- Par conséquent, nous utilisons les connaissances de **code source** définir les cas de test et examiner les résultats.
- Les cas de test conçus pour tester l'ensemble de base garantissent l'exécution de chaque instruction du programme au moins une fois pendant les tests.

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

Qu'est-ce que Path ?

- Un chemin d'accès dans un programme est une séquence d'instructions qui commence à un point d'entrée, une jonction ou une décision et se termine.
- Un chemin peut passer par plusieurs carrefours, processus ou décisions, une ou plusieurs fois.
- Les chemins sont constitués de segments ; le plus petit segment est un lien entre deux nœuds.

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

Qu'est-ce que Path ?

- **Un chemin** Un programme est une séquence d'instructions ou d'énoncés qui commence à une entrée, une jonction ou une décision et se termine à une autre, ou éventuellement à la même jonction, décision ou sortie.
- **Répétition** : Un chemin peut passer par plusieurs jonctions, processus ou décisions, une ou plusieurs fois.
- **Composition** : Les chemins sont constitués de segments.
- **Le segment** : un segment est un lien — un processus unique qui se situe entre deux nœuds.

Méthodologies d'essai : Techniques d'essais structuraux

ESSAIS DE DÉBIT RÉGULATEUR

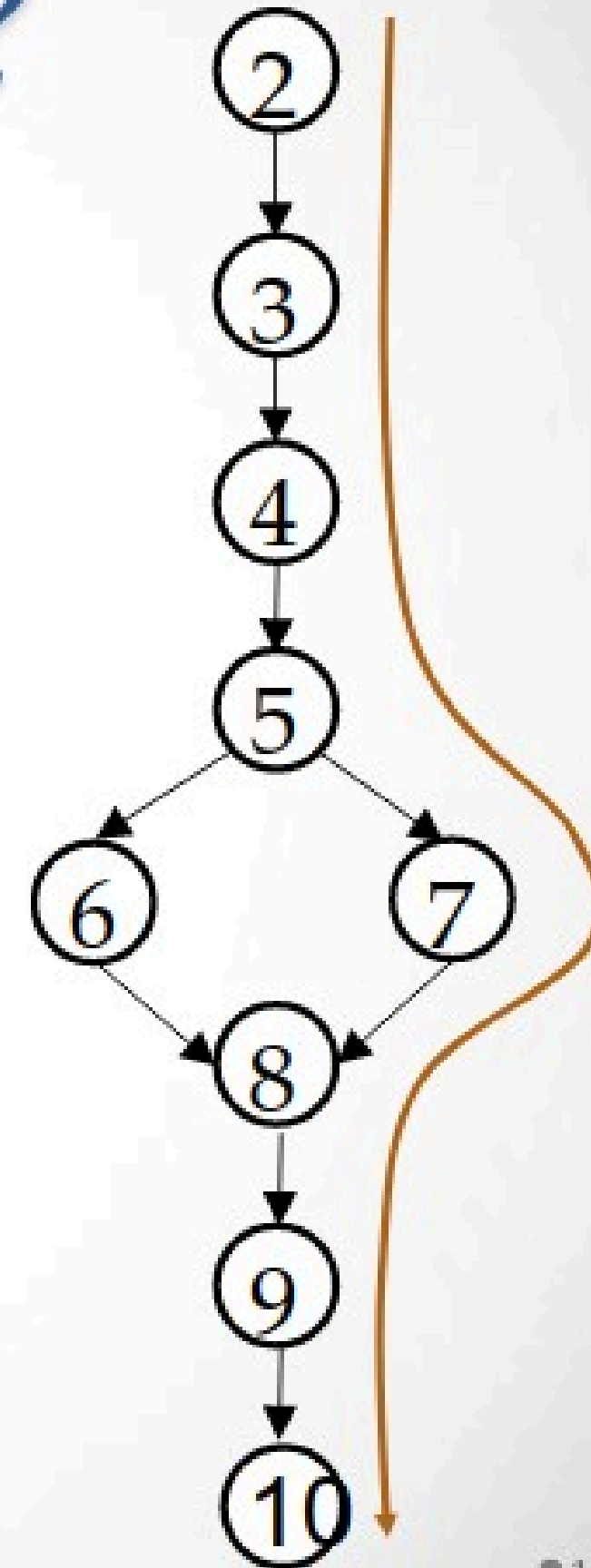
Qu'est-ce que Path ?

- **Segment de chemin** : Un segment de chemin est une succession de liens consécutifs appartenant à un chemin donné.
- **Mesure (Longueur)** : La longueur d'un chemin est mesurée par le nombre de liens qui le composent, et non par le nombre d'instructions ou d'instructions exécutées le long de ce chemin.
- **Nommage** : Le nom d'un chemin correspond au nom des nœuds qui le jalonnent.

ESSAIS DE DÉBIT RÉGULATEUR

What is Path?

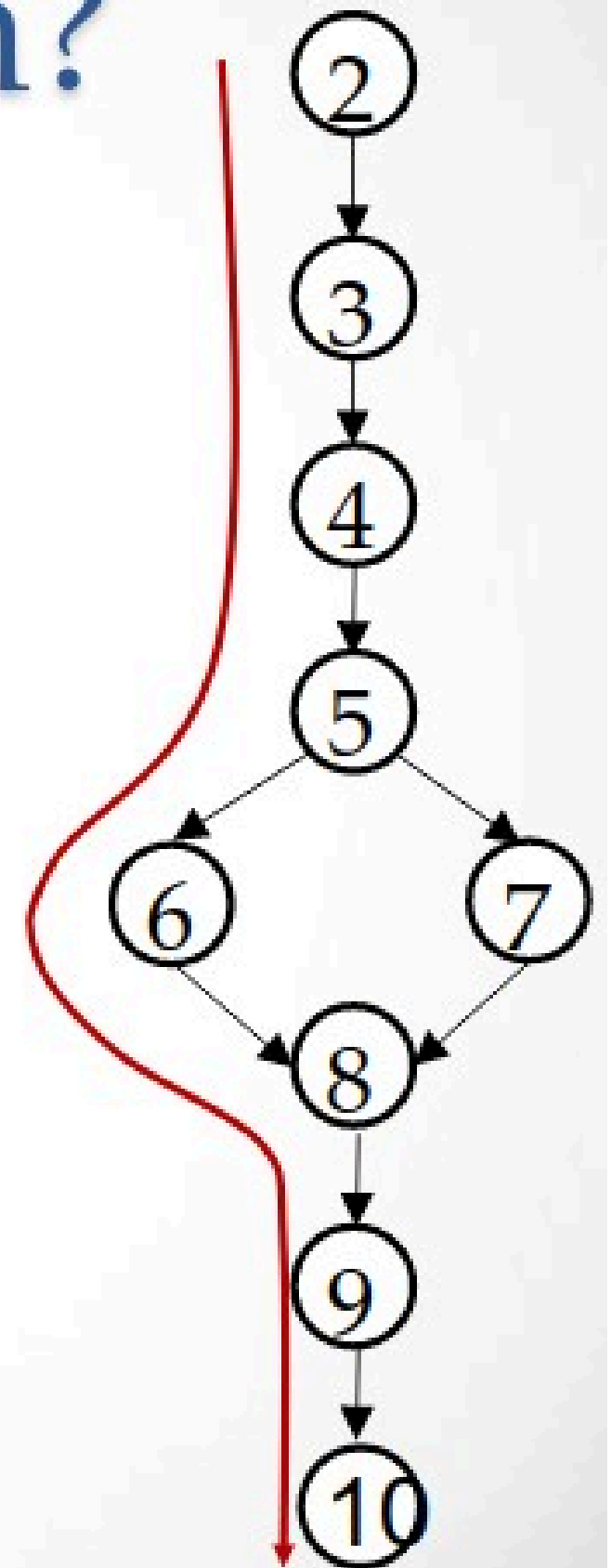
- For example:
Path2 = 2-3-4-5-7-8-9-10



ESSAIS DE DÉBIT RÉGULATEUR

What is Path?

- For example:
Path1 = 2-3-4-5-6-8-9-10



Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

CRITÈRES FONDAMENTAUX DE SÉLECTION DE LA VOIE :

Il existe de nombreux chemins entre l'entrée et la sortie d'une routine typique.

- Chaque décision **double le nombre** des voies potentielles.
- Chaque boucle **multiplie le nombre** des chemins potentiels en fonction du nombre de valeurs d'itération différentes possibles pour la boucle.

Dans une tentative de définir des tests complets : 0.

Parcourez tous les chemins de l'entrée à la sortie.

1. Mettez en pratique chaque consigne ou instruction au moins une fois.
2. Exécutez chaque instruction de branchement et chaque instruction de cas, dans chaque sens au moins une fois.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

CRITÈRES DES STRATÉGIES DE TEST DE CHEMINÉE :

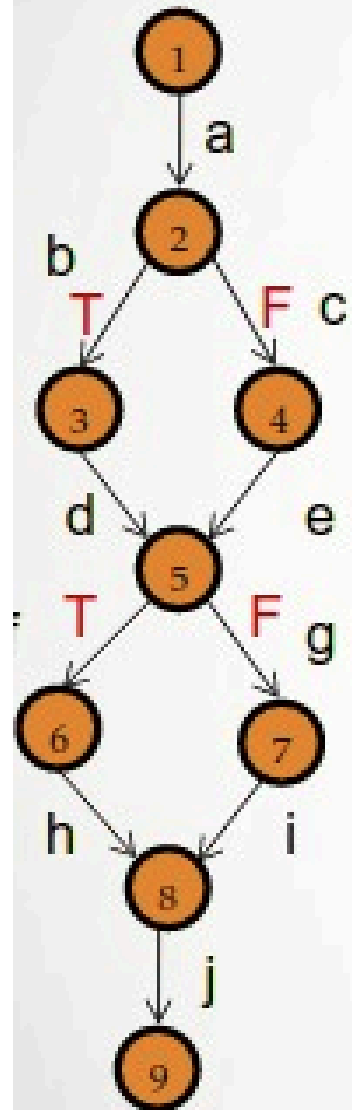
Toute stratégie de test basée sur des chemins doit au moins tester chaque instruction et explorer toutes les directions.

Un ensemble de tests qui réalise cela n'est pas complet au sens absolu, mais il est complet en ce sens que tout ensemble inférieur doit laisser quelque chose non testé.

Nous allons donc explorer trois critères ou stratégies de test différents parmi une famille potentiellement infinie de stratégies.

- 1. Tests de déclaration/Couverture de déclaration**
- 2. Tests de succursales / Couverture des succursales**
- 3. Tests de chemin / Couverture de chemin**

Path Testing Strategies



- **Statement Testing**
 - 100% statement / node coverage
- **Branch Testing**
 - 100% branch/ link coverage
- **Path Testing**
 - 100% path coverage

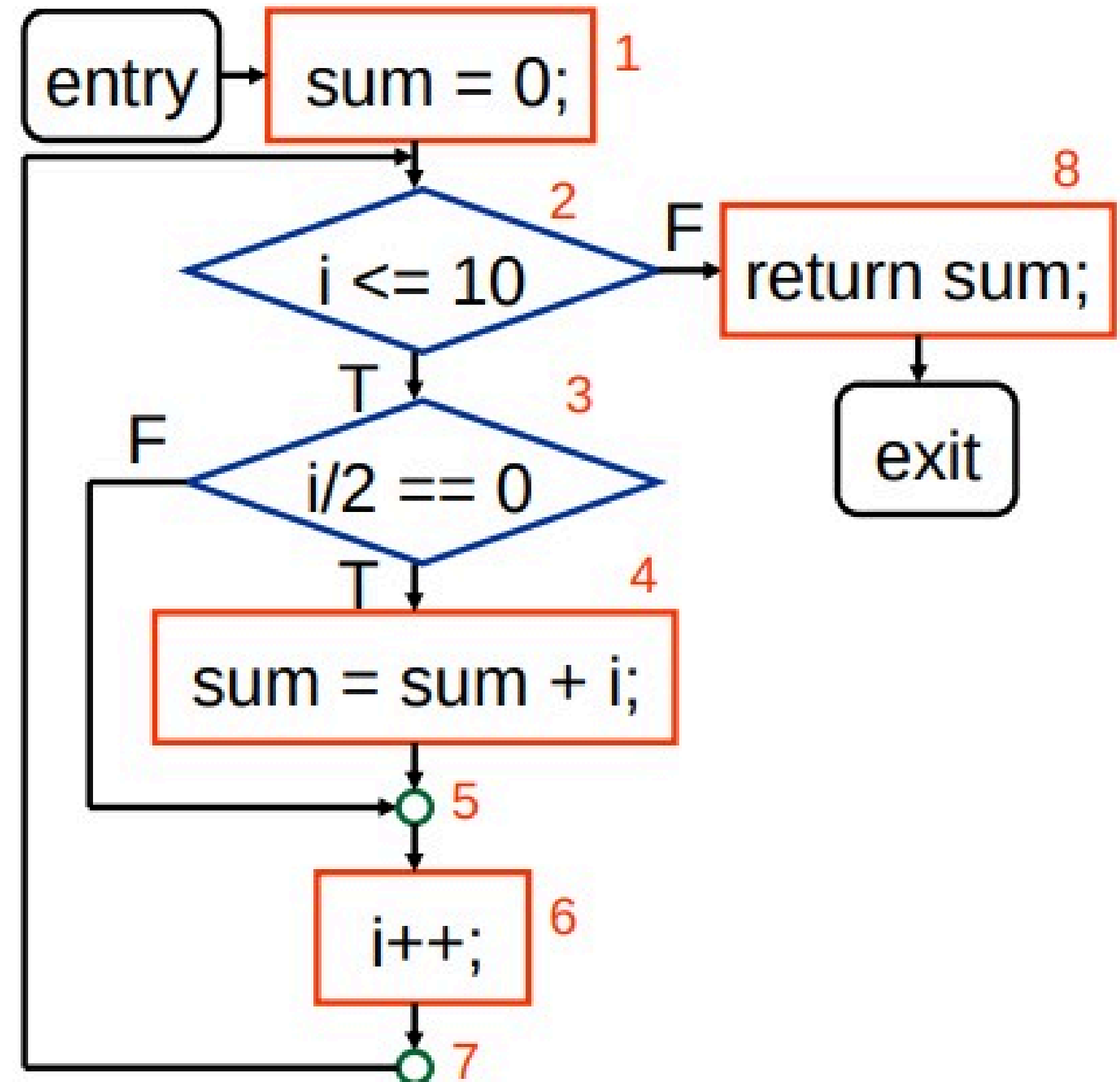
- **Déclaration essai ou déclaration**
La couverture de code consiste à exécuter toutes les instructions du programme au moins une fois lors d'un test donné. Si nous effectuons suffisamment de tests pour y parvenir, on dit que nous avons atteint une couverture de code de 100 %.
- **Une autre formulation équivalente consiste à dire que nous avons atteint 100%. couverture des nœuds.**
- **Désignons cela par C1.**
- **Il s'agit du critère le moins contraignant de la famille : tester moins que cela pour les nouveaux logiciels est déraisonnable (sans principes ou inacceptable) et devrait être criminalisé (inacceptable).**

Statement Coverage

TESTS DE CHEMINEMENT

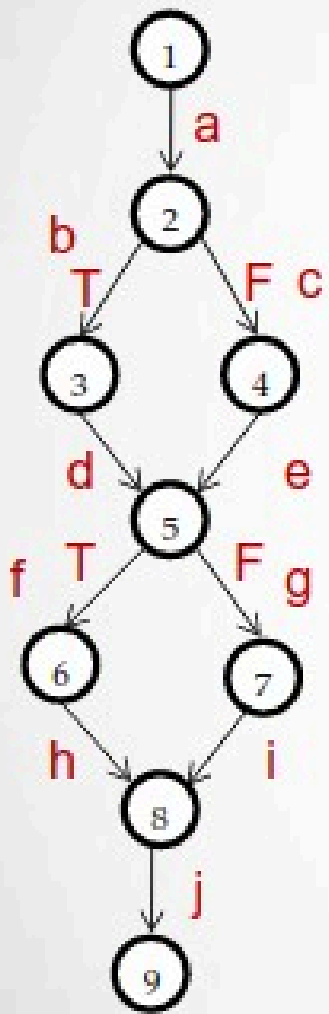
- Every **statement** in the program has been executed at least once.

1 → 2 → 3 → 4 →
5 → 6 → 7 → 2 → 8



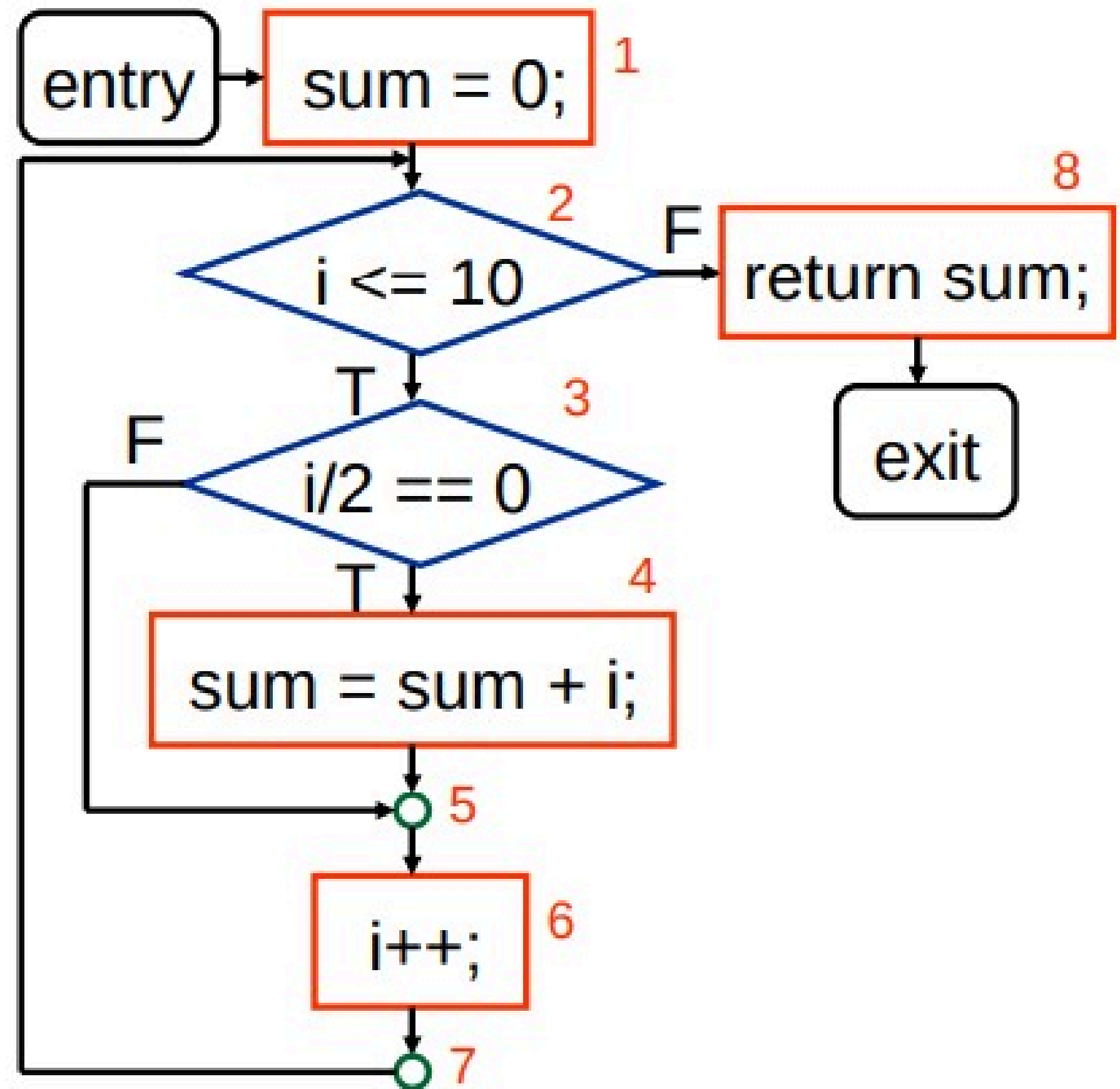
Path Testing Strategies

- Statement Testing
 - 100% statement / node coverage
- Branch Testing
 - 100% branch/ link coverage
- Path Testing
 - 100% path coverage



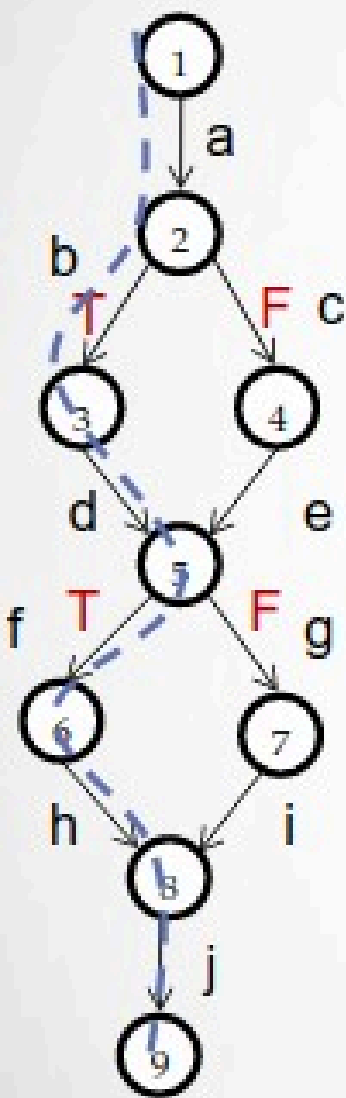
- Les tests de branchement/la couverture de branchement/ les tests de condition exécutent suffisamment de tests pour garantir que chaque alternative de branchement a été testée au moins une fois dans le cadre d'un test donné. Autrement dit, les tests de branchement exécutent simultanément chaque instruction et chaque condition dans le cadre d'un test donné.
- Si nous effectuons suffisamment de tests pour parvenir à cette prescription, alors nous aurons atteint une couverture de 100 % des succursales.
- On peut aussi dire que nous avons atteint une couverture de liens de 100 %. Pour les logiciels structurés, les tests de branchement, et donc la couverture de branchement, incluent strictement la couverture des instructions.
- Désignons la couverture de branche par C2.

- Every **statement** in the program has been executed at least once, and every **decision** in the program has taken all possible outcomes at least once.



1 → 2 → 3 → 5 → 6 → 7 → 2 →
3 → 4 → 5 → 6 → 7 → 2 → 8

Path Testing Strategies



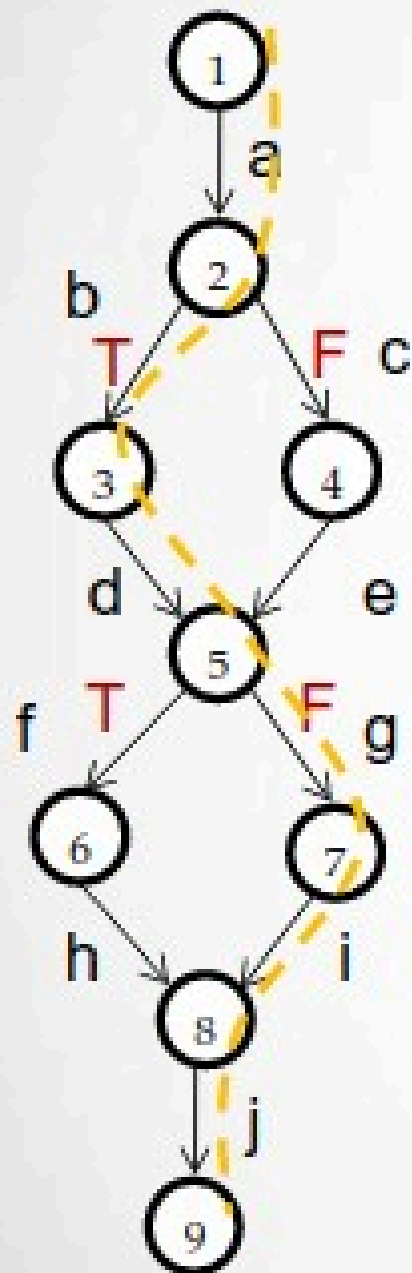
- Statement Testing
 - 100% statement / node coverage
- Branch Testing
 - 100% branch/ link coverage
- Path Testing
 - 100% path coverage

Statement Testing < Branch Testing < Path Testing

- Les tests de chemin ou la couverture de chemin consistent à exécuter au moins une fois tous les chemins d'exécution possibles du flux de contrôle à travers le programme : généralement, cela se limite à tous les points d'entrée/sortie possibles. Si nous
- respectons cette condition, on dit que nous avons atteint une couverture de chemin de 100 %, ce qui est pratiquement impossible en raison du problème inhérent de...**explosion de trajectoire**.
- **Il s'agit du critère le plus fort** dans la famille des stratégies de test de chemin : c'est généralement impossible à réaliser.

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Path Testing Strategies

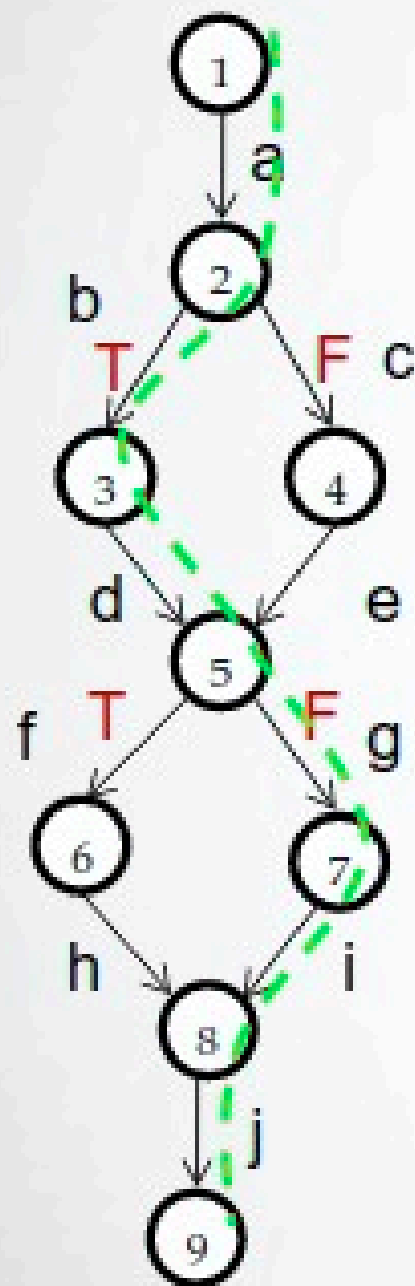


- Statement Testing
 - 100% statement / node coverage
- Branch Testing
 - 100% branch/ link coverage
- Path Testing
 - 100% path coverage

Statement Testing < Branch Testing < Path Testing

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Path Testing Strategies

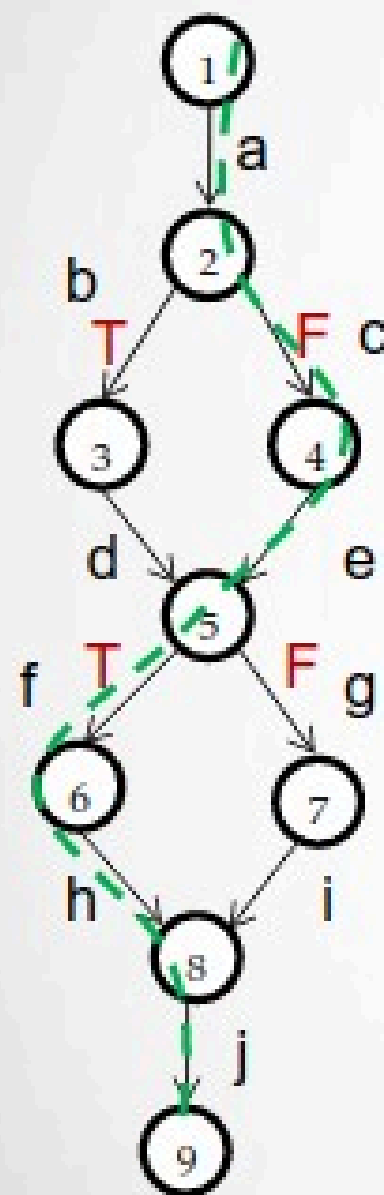


- Statement Testing
 - 100% statement / node coverage
- Branch Testing
 - 100% branch/ link coverage
- Path Testing
 - 100% path coverage

Statement Testing < Branch Testing < Path Testing

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Path Testing Strategies



- Statement Testing
 - 100% statement / node coverage
- Branch Testing
 - 100% branch/ link coverage
- Path Testing
 - 100% path coverage

Statement Testing < Branch Testing < Path Testing

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Quelques conseils et stratégies : La question « pourquoi ne pas utiliser un échantillonnage judicieux de chemins ? »

- La couverture des succursales et des relevés est aujourd'hui acceptée comme l'exigence minimale de test obligatoire.

La question « **Pourquoi ne pas utiliser un échantillon judicieux de chemins ?? Quel est le problème à laisser une partie du code non testée ?** » est inefficace du point de vue du bon sens et de l'expérience puisque :

(1.) Ne pas tester un morceau de code laisse un résidu de bogues dans le programme proportionnel à la taille du code non testé et à la probabilité de bogues.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Quelques conseils et stratégies : La question « pourquoi ne pas utiliser un échantillonnage judicieux de chemins ? »

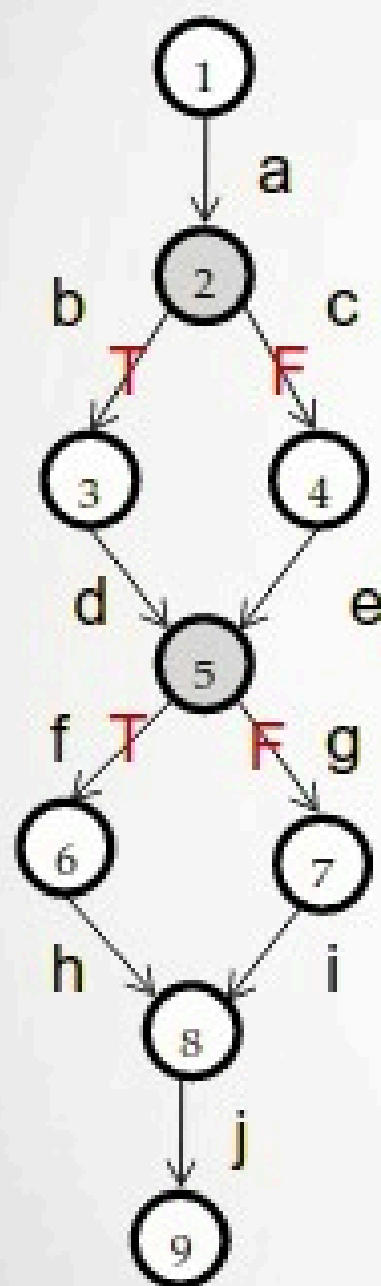
(2.) Les chemins à forte probabilité sont toujours testés de manière approfondie, ne serait-ce que pour démontrer que le système fonctionne correctement.

- **Quels chemins faut-il tester ? =====>>C1+C2.**
- **Vous devez choisir suffisamment de chemins pour atteindre C1+C2** La question de savoir quel est le nombre minimal de tels chemins est intéressante pour le concepteur d'outils de test qui aident à automatiser les tests de chemin, mais elle n'est pas cruciale pour la conception pragmatique (pratique) des tests.
- **Il vaut mieux tracer de nombreux chemins simples que quelques chemins compliqués.**

Méthodologies d'essai : Techniques d'essais structuraux

A Coverage Table

TESTS DE RÉGULATION DU DÉBIT :
TESTS DE CHEMINEMENT



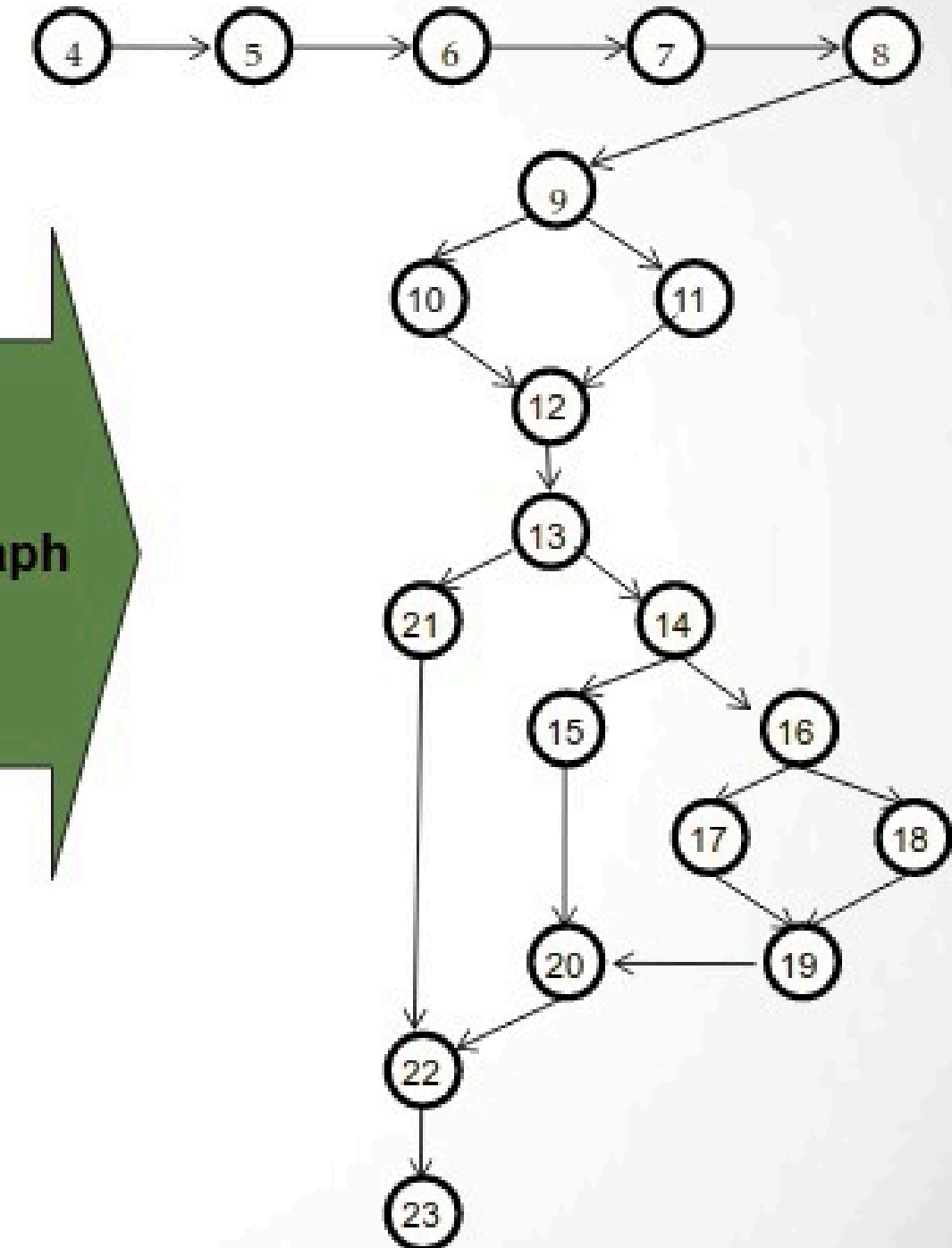
Paths	Decisions		Process-Link									
	2	5	a	b	c	d	e	f	g	h	i	j
1-2-3-5-6-8-9 (a-b-d-f-h-j)	T	T	✓	✓		✓		✓		✓		✓
1-2-4-5-7-8-9 (a-c-e-g-i-j)	F	F	✓		✓		✓		✓		✓	✓
1-2-3-5-7-8-9 (a-b-d-g-i-j)	T	F	✓	✓		✓			✓		✓	✓
1-2-4-5-6-8-9 (a-c-e-f-h-j)	F	T	✓		✓		✓	✓		✓		✓

TESTS DE CONTRÔLE DE FLUX : TESTS DE TRAJET

Exemple Flow Graph

1. Program Triangle
2. Dim a, b, c As Integer
3. Dim IsTriangle As Boolean
4. Output ("enter a,b, and c integers")
5. Input (a,b,c)
6. Output ("side 1 is", a)
7. Output ("side 2 is", b)
8. Output ("side 3 is", c)
9. If (a<b+c) AND (b<a+c) And (c<b+a)
10. then IsTriangle = True
11. else IsTriangle = False
12. endif
13. If IsTriangle
14. then if (a=b) AND (b=c)
15. then Output ("equilateral")
16. else if (a != b) AND (a != b) AND (b != c)
17. then Output ("Scalene")
18. else Output ("Isosceles")
19. endif
20. endif
21. else Output ("not a triangle")
22. endif
23. end Triangle2

Flow Graph



FLUX DE CONTRÔLE ESSAI

Test Case for Path Coverage

- ① 4-5-6-7-8-9-10-12-13-21-22-23
- ② 4-5-6-7-8-9-11-12-13-14-15-20-22-23
- ③ 4-5-6-7-8-9-11-12-13-14-16-17-19-20-22-23
- ④ 4-5-6-7-8-9-11-12-13-14-16-18-19-20-22-23

A Coverage Table

Path	Decision				Test Case			Expected Results
	9	13	14	16	a	b	c	
1	T	F			100	100	200	Not a Triangle
2	F	T	T		60	60	60	Equilateral
3	F	T	F	T	50	60	70	Scalene
4	F	T	T	F	75	75	30	Isosceles

EXERCICE UN (20 points)

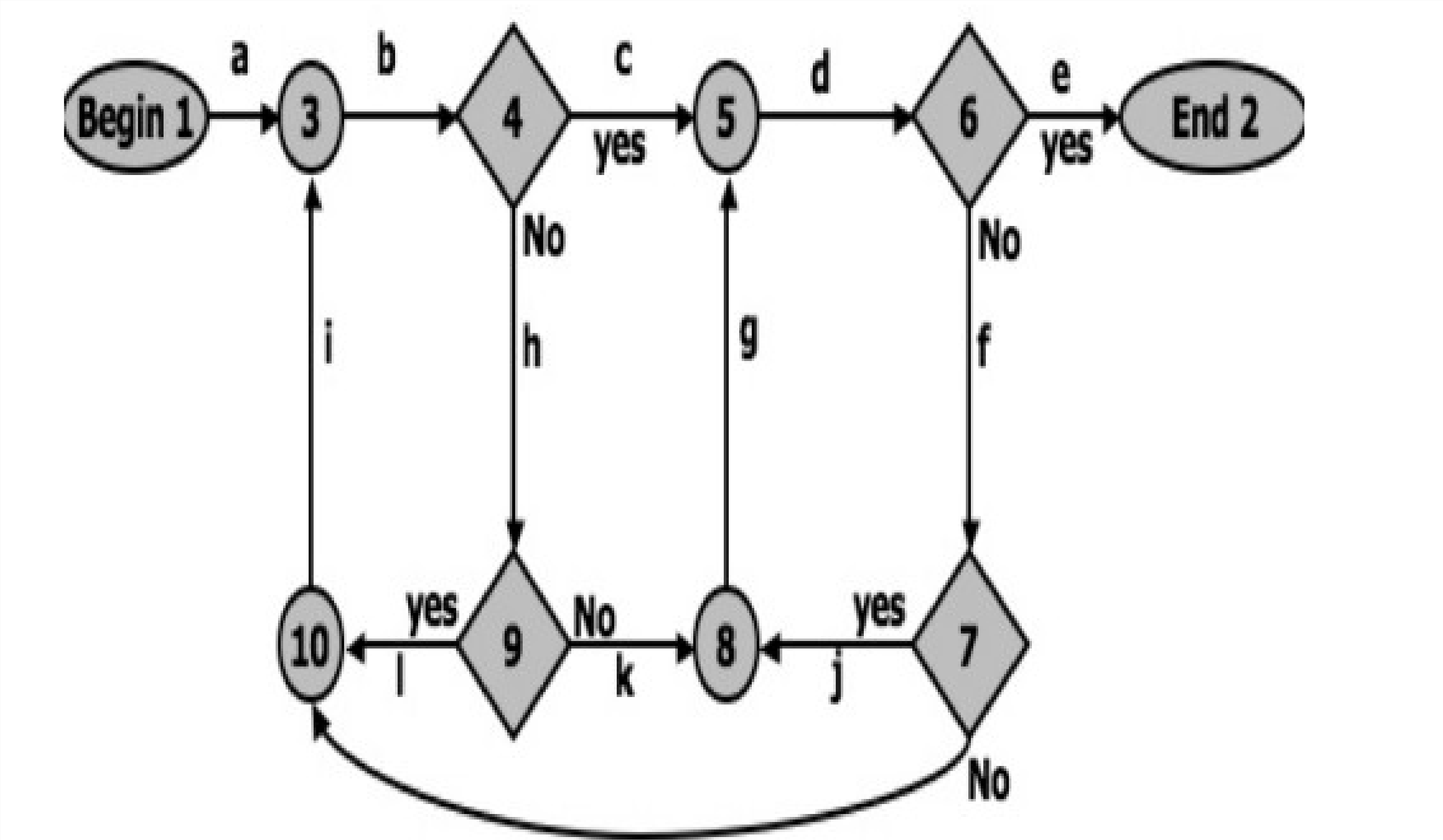
(1) Supposons que vous deviez développer un logiciel pour la gestion des résultats des étudiants à l'Université de Yaoundé I avec le système de notation suivant :

Grade	Plage (xx)
A+	$80 \leq x \leq 100$
UN	$75 \leq x < 80$
B+	$70 \leq x < 75$
B	$65 \leq x < 70$
B-	$60 \leq x < 65$
C+	$55 \leq x < 60$
C	$50 \leq x < 55$
C-	$45 \leq x < 50$
D+	$40 \leq x < 45$
D	$35 \leq x < 40$
E	$0 \leq x < 35$

EXERCICE UN (20 points)

- (a) Écrivez un algorithme pour le projet logiciel ci-dessus et indiquez ensuite les caractéristiques des algorithmes ?**
- (b) Définir les cas de test et expliquer les critères de sélection de ces cas de test.**
- (c) Présenter le document de cas de test du projet choisi**
- d) À l'aide de Java NetBeans, écrivez la fonction permettant de calculer les notes de N étudiants et représentez le graphe de flux de contrôle (CFG) du système (dans le terminal et dans l'interface graphique).**
- e) En lien avec le point d), expliquez en détail :**
 - Tests disruptifs et tests exhaustifs**
 - Tests de flux de données (All-Defs, All-Uses, All-DU-Paths, All-P-Uses, Some-C-Uses, All-C-Uses, Some-P-Uses, All-P-Uses, All-C-Uses)**
 - Analyse de chemin – Mesure de la complexité cyclomatique et sélection de chemin**
- (f) Identifiez cinq chemins dans (d) et produisez ainsi le tableau de couverture, en indiquant explicitement les entrées valides et invalides.**
- (g) Recommandez trois stratégies de test de chemin pour le système en donnant votre justification et l'importance de chaque stratégie.**

Exercice deux :Identifiez cinq chemins dans le CFG et produisez ainsi le tableau de couverture.



Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Définition:

- **Tests de flux de données** Le test de fonctionnalité dynamique (DFT) est une technique de test en boîte blanche qui examine la définition et l'utilisation des variables dans votre code. C'est une méthode puissante pour déceler les bogues en traçant le parcours des données au sein de votre programme.
- Contrairement à tests de boîte noire, qui ne prend en compte que les entrées et les sorties, **tests de flux de données** vous offre une visibilité sur la logique interne et la structure du code.
- Le test de la fonctionnelle de la densité (DFT) est devenu essentiel car il permet de déceler les problèmes et les faiblesses cachés dans la circulation des données au sein des logiciels. Les chiffres montrent que les problèmes liés au flux de données restent une préoccupation majeure pour les développeurs et les entreprises.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

problèmes courants :

Lorsque les variables et leurs valeurs interagissent au sein d'un programme, les tests de flux de données permettent de déceler trois problèmes courants :

- Une variable est définie mais jamais utilisée ni référencée dans le code. Une
- variable est utilisée avant même d'être définie.
- Une variable est définie plusieurs fois avant d'être utilisée.

En générant cas de test qui couvrent les chemins de flux de contrôle autour de ces paires d'utilisation de définition, **tests de flux de données** garantit que les données circulent dans vos modules exactement comme prévu.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Points clés à retenir :

Tests de flux de données Cette méthode s'intéresse à la définition, à l'utilisation et à la gestion des variables au sein d'un programme. En traçant les paires définition-utilisation, elle permet de déceler des défauts de données cachés que les tests logiques traditionnels peuvent négliger.

Comment utiliser les tests de flux de données ?

Étape 1 : Analyser le code du programme

Commencez par parcourir le code source pour identifier chaque point de définition (affectation d'une valeur) et d'utilisation des variables. Vous pouvez le faire manuellement en lisant le code, ou accélérer le processus à l'aide d'outils d'analyse statique comme SonarQube, SpotBugs, PMD ou CodeClimate. Ces outils analysent votre code et génèrent des rapports détaillés indiquant précisément où les variables sont déclarées, affectées et référencées.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Comment utiliser les tests de flux de données ?

Étape 2 : Créer un graphe de flux de données

Une fois que vous avez défini les définitions et les utilisations, créez un graphique de flux de données pour visualiser comment les données circulent dans le programme.

Chaque nœud du graphe représente un point où une variable est définie ou utilisée, et chaque arête représente le chemin entre ces points.

Cela facilite grandement le repérage des zones à problèmes potentiels. Vous pouvez créer ces graphiques manuellement ou utiliser des outils comme **Générateurs CFG**, **Paradigme visuel**, ou tout outil d'édition de graphiques correspondant.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Comment utiliser les tests de flux de données ?

Étape 3 : Identifier les chaînes de définition-utilisation (DU)

Il s'agit de l'étape la plus cruciale. Une chaîne DU retrace le chemin d'accès d'une variable, de sa définition à son utilisation, sans qu'elle soit redéfinie entre-temps.

Examinez attentivement chaque chaîne pour confirmer que chaque définition de variable atteint correctement son usage prévu.

Des outils comme FlowDroid et CodeSonar peuvent aider à automatiser cette analyse, notamment dans les bases de code plus importantes.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Comment utiliser les tests de flux de données ?

Étape 4 : Créer et exécuter des cas de test

Enfin, utilisez les chaînes DU pour concevoir des cas de test couvrant tous les parcours possibles d'une variable dans le programme. Chaque test doit vérifier que la variable se comporte comme prévu d'après sa définition. Des frameworks comme JUnit (Java), NUnit (.NET), ou TestNG vous aider à automatiser et à exécuter efficacement ces tests.

Points clés à retenir : **Candidature tests de flux de données** Cela implique d'analyser les définitions des variables, de cartographier leur flux, d'identifier les chaînes définition-utilisation et de concevoir des cas de test ciblés. Cette approche structurée garantit la bonne circulation des données dans votre application et prévient les erreurs d'exécution subtiles.

Méthodologies d'essai : Techniques d'essais structuraux

TESTS DE FLUX DE DONNÉES

Exemple de test de flux de données

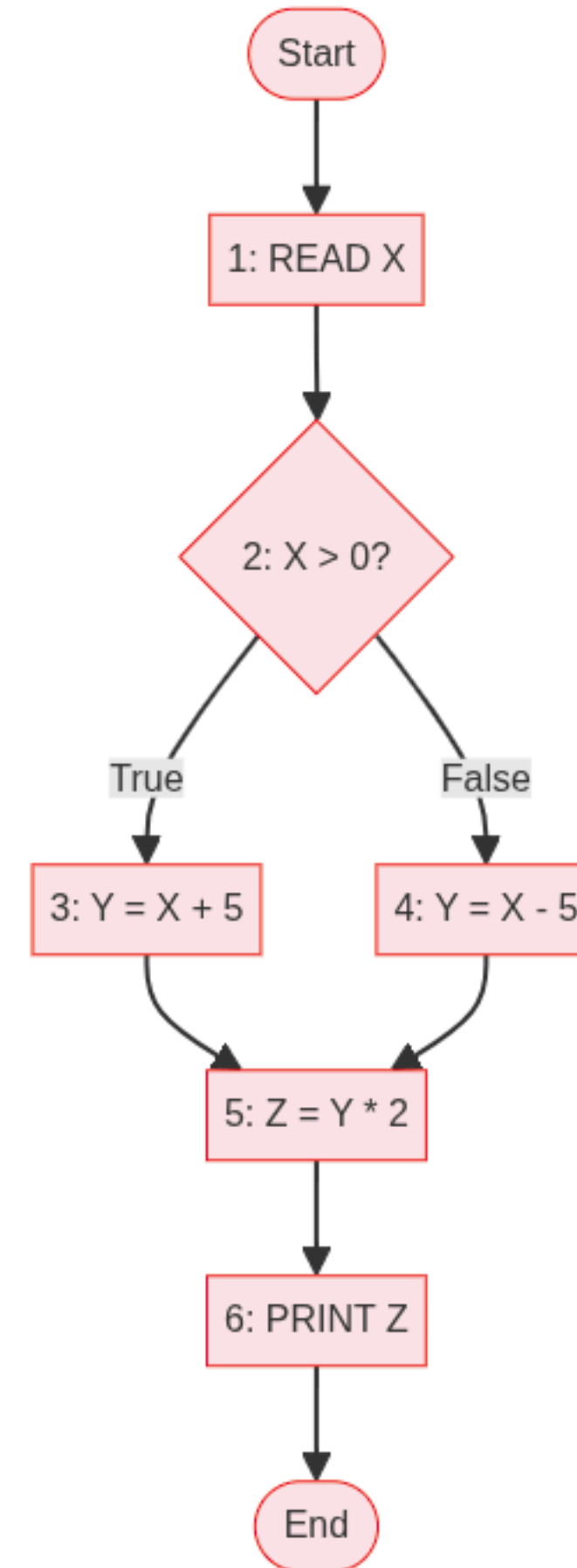
Prenons comme exemple le bloc de code ci-dessous, à partir duquel nous créerions le graphe de flux de contrôle, puis évaluerions les tests de flux de données.

```
1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          // Create a scanner object for user input (READ X)
6          Scanner scanner = new Scanner(System.in);
7
8          System.out.print("Enter the value for X: ");
9          double x = scanner.nextDouble();
10         double y;
11
12         // Conditional logic (IF X > 0 ...)
13         if (x > 0) {
14             y = x + 5; // THEN Y = X + 5
15         } else {
16             y = x - 5; // ELSE Y = X - 5
17         }
18
19         // Calculation (Z = Y * 2)
20         double z = y * 2;
21
22         // Output the result (PRINT Z)
23         System.out.println("The value of Z is: " + z);
24
25         scanner.close();
26     }
27 }
```


Méthodologies d'essai : Techniques d'essais structuraux

Exemple de test de flux de données

Le diagramme de flux de contrôle des lignes de code ci-dessus est décrit ci-dessous :



Module III : Application des niveaux de test logiciel